



École Polytechnique Fédérale de Lausanne

A wonderful thesis
about the merits of scientific writing

by Antonino Emanuele Scurria

Master Thesis

Approved by the Examining Committee:

Prof. Mark Seidel
Thesis Advisor

a
External Expert

Laurent Deniau
Thesis Supervisor

EPFL IC IINFCOM HEXHIVE
BC 160 (Bâtiment BC)
Station 14
CH-1015 Lausanne

December 6, 2024

Follow the white rabbit...
— The Matrix

Dedicated to my pet bunny.

The dedication is usually a short inspirational quote.

Define your dedication in `\dedication{...}` and show them with `\makededication`.

Acknowledgments

This is where you thank those who supported you on this journey. Good examples are your significant other, family, advisers, and other parties that inspired you during this project. Generally this section is about 1/2 page to a page.

Consider acknowledging the use and location of this thesis package.

Define your acknowledgments in `\acknowledgments{...}` and show them with `\makeacks`.

Lausanne, December 6, 2024

Antonino Emanuele Scurria

Abstract

The FooSystem tool enables lateral decomposition of a multi-dimensional flux compensator along the timing and space axes.

The abstract serves as an executive summary of your project. Your abstract should cover at least the following topics, 1-2 sentences for each: what area you are in, the problem you focus on, why existing work is insufficient, what the high-level intuition of your work is, maybe a neat design or implementation decision, and key results of your evaluation.

Résumé

For a doctoral thesis, you have to provide a French translation of the English abstract. For other projects this is optional.

Contents

Acknowledgments	1
Abstract (English/Français)	2
1 Background	6
1.1 Overview of Dual Numbers	6
1.2 Algebraic Properties of Dual Numbers	6
1.3 Differentiation Using Dual Numbers	8
1.3.1 Multivariate dual numbers	10
1.4 The Importance of Dual Numbers in Accelerator Physics	12
1.4.1 The Computation of Transfer Maps	13
1.4.2 Numerical maps approach	15
1.4.3 Real value of TPSA in Accelerator physics	15
1.5 Applications of Dual Numbers Beyond Accelerator Physics	17
2 Enhancing the TPSA library in MAD-NG	19
2.1 Motivation	19
2.2 Atan, Acot, Atanh, Acoth	19
2.3 Acosh	28
2.4 Faddeeva function	29
2.5 Sinc and Sinh c	33
2.6 Asinc and Asinh c	34
3 Code and Results	36
3.1 Data Generation and Testing Framework	36
3.1.1 Data Generation	36
3.1.2 Sympy code	37
3.1.3 Performance Analysis	40
3.2 Numerical implementation and Performance	41
3.2.1 Atan	42
3.2.2 Acot, Atanh and Acoth	46
3.2.3 Acosh	47

3.2.4	Faddeeva function	49
3.2.5	Sinc	57
3.2.6	Sinhc	63
3.2.7	Asinc	64
3.2.8	Asinhc	69
4	Case Study	70
4.1	Beam-Beam effect and Twiss parameters	70
4.2	Beam-Beam Effect	70
4.2.1	Electromagnetic Interaction	71
4.2.2	Beam-Beam Force	71
4.2.3	Beam-Beam Parameter	72
4.2.4	Impact on Beam Dynamics	72
4.2.5	Faddeeva Function in Beam-Beam Interactions	73
4.3	Twiss Parameters	73
4.3.1	Numerical Result	75
4.4	Summary	79
A	Appendix	80
A.1	Data Generation Code	80

Chapter 1

Background

1.1 Overview of Dual Numbers

Dual numbers form a mathematical extension of the real numbers that allows for the representation of infinitesimal quantities. They are defined using a dual unit ϵ with the property $\epsilon^2 = 0$ and $\epsilon \neq 0$. The dual numbers have applications in fields such as differential calculus, kinematics, and automatic differentiation, where they facilitate calculations of derivatives in an elegant and efficient manner.

Definition 1.1.1. A *dual number* x is an element of the form

$$x = a + b\epsilon,$$

where $a, b \in \mathbb{C}$, and ϵ is a unit with the property $\epsilon^2 = 0$ and $\epsilon \neq 0$. The real number a is called the *scalar part* of x , and b is called the *dual part*.

The set of all dual numbers is denoted by $\mathbb{D} = \{a + b\epsilon \mid a, b \in \mathbb{C}\}$, and forms a commutative ring under the usual operations of addition and multiplication.

1.2 Algebraic Properties of Dual Numbers

We define addition, multiplication, and conjugation in the ring $\mathbb{D}$ and prove several properties.

Definition 1.2.1. Let $x = a + b\epsilon$ and $y = c + d\epsilon$ be two dual numbers in $\mathbb{D}$.

- The *sum* of x and y is given by

$$x + y = (a + c) + (b + d)\epsilon.$$

- **Scalar Multiplication:** For any scalar $c \in \mathbb{C}$, the scalar multiplication of c with x is

$$c \cdot x = (c \cdot a, c \cdot b).$$

- The **product** of x and y is given by

$$x \cdot y = (a + b\epsilon)(c + d\epsilon) = ac + (ad + bc)\epsilon,$$

where we use the fact that $\epsilon^2 = 0$.

- The **conjugation** of a dual number x is defined as

$$\bar{x} = a - b\epsilon.$$

Proposition 1. The set $\mathbb{D}$ of dual numbers is a commutative ring with respect to addition and multiplication.

Proof. To prove commutativity, associativity, and distributivity, we consider the operations as defined above and verify that they satisfy the axioms of a commutative ring. For example:

$$x + y = (a + c) + (b + d)\epsilon = (c + a) + (d + b)\epsilon = y + x,$$

and

$$x \cdot y = ac + (ad + bc)\epsilon = ca + (cb + da)\epsilon = y \cdot x.$$

The remaining axioms are verified similarly. □

Proposition 2. For any dual number $x = a + b\epsilon$, the conjugate satisfies

$$x \cdot x^* = a^2.$$

Proof. Since $x^* = a - b\epsilon$, we have

$$x \cdot x^* = (a + b\epsilon)(a - b\epsilon) = a^2 - ab\epsilon + ab\epsilon - b^2\epsilon^2 = a^2.$$

The terms involving ϵ cancel out, and $b^2\epsilon^2 = 0$. □

Proposition 3. If $x = a + b\epsilon$ with $a \neq 0$, the inverse of x exists and is given by

$$x^{-1} = \frac{1}{a} - \frac{b}{a^2}\epsilon.$$

Proof. We seek $y = c + d\epsilon$ such that $x \cdot y = 1$. Expanding $x \cdot y$, we find:

$$(a + b\epsilon)(c + d\epsilon) = ac + (ad + bc)\epsilon.$$

Setting the real and dual parts equal to those of $1 + 0\epsilon$, we obtain $ac = 1$ and $ad + bc = 0$. Solving these equations yields $c = \frac{1}{a}$ and $d = -\frac{b}{a^2}$, giving the result. $\square$

1.3 Differentiation Using Dual Numbers

A notable application of dual numbers is in automatic differentiation. By evaluating functions at dual number inputs, we can obtain both the function value and its derivative simultaneously.

Definition 1.3.1. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a differentiable function, and let $x = a + \epsilon$ be a dual number. Then the *dual extension* of f evaluated at x is given by

$$f(a + \epsilon) = f(a) + f'(a)\epsilon.$$

Proof. Expanding f in a Taylor series around a and substituting $a + \epsilon$ for x , we have:

$$f(a + \epsilon) = f(a) + f'(a) \cdot \epsilon + \frac{f''(a) \cdot \epsilon^2}{2!} + \dots.$$

Since $\epsilon^2 = 0$, all terms of order ϵ^2 and higher vanish, leaving

$$f(a + \epsilon) = f(a) + f'(a)\epsilon.$$

$\square$

Example 1. Let $f(x) = x^2$. To find $f'(3)$ using dual numbers, consider $x = 3 + \epsilon$:

$$f(3 + \epsilon) = (3 + \epsilon)^2 = 9 + 6\epsilon.$$

Thus, $f(3) = 9$ and $f'(3) = 6$.

Higher-Order Dual Numbers

To extend the concept of dual numbers to higher orders, we generalize the notion of infinitesimal terms to represent higher derivatives. For a given order N , a higher-order dual number encodes information about a function and its derivatives up to the N -th order. This extension is beneficial for automatic differentiation, Taylor series expansions, and computations involving higher-order partial derivatives in multivariable functions.

Definition 1.3.2. Let $N \in \mathbb{N}$ be the order of the dual number. A higher-order dual number q of order N is an ordered tuple:

$$q = \sum_{i=0}^N q_i \epsilon^i =: (q_0, q_1, q_2, \dots, q_N),$$

where $q_0 \in \mathbb{C}$ represents the scalar component, and $q_1, q_2, \dots, q_N$ are coefficients associated with infinitesimal terms representing the first, second, and higher-order derivatives up to the N -th order.

The algebra of higher-order dual numbers involves component-wise definitions for addition and multiplication, which are consistent with the rules for derivatives.

Definition 1.3.3. Let $q = (q_0, q_1, \dots, q_N)$ and $r = (r_0, r_1, \dots, r_N)$ be two higher-order dual numbers of order N .

- **Addition:** The sum of q and r is given by

$$q + r = (q_0 + r_0, q_1 + r_1, \dots, q_N + r_N).$$

- **Scalar Multiplication:** For any scalar $c \in \mathbb{R}$, the scalar multiplication of c with q is

$$c \cdot q = (c \cdot q_0, c \cdot q_1, \dots, c \cdot q_N).$$

- **Multiplication:** The product of q and r is defined component-wise by

$$(q \cdot r)_i = \sum_{k=0}^i \binom{i}{k} q_k r_{i-k}, \quad i = 0, 1, \dots, N,$$

where $\binom{i}{k} = \frac{i!}{k!(i-k)!}$ is the binomial coefficient.

This multiplication rule generalizes the product rule for derivatives. For instance, in the case of second-order dual numbers, where $q = (q_0, q_1, q_2)$ and $r = (r_0, r_1, r_2)$, the product components are computed as:

$$\begin{aligned} s_0 &= q_0 r_0, \\ s_1 &= q_0 r_1 + q_1 r_0, \\ s_2 &= q_0 r_2 + q_1 r_1 + q_2 r_0. \end{aligned}$$

Each term s_i in the product $s = q \cdot r = (s_0, s_1, \dots, s_N)$ incorporates the appropriate derivative terms according to the Leibniz rule.

1.3.1 Multivariate dual numbers

Here we briefly describe dual numbers in the multivariate case: we are going to use an analogy of this kind between the dual numbers and monomials

$$c + \alpha_x \epsilon_x + \alpha_y \epsilon_y + \alpha_{x^2} \epsilon_x^2 + \alpha_{xy} \epsilon_x \epsilon_y + \alpha_{y^2} \epsilon_y^2 \rightarrow c + \alpha_x x + \alpha_y y + \alpha_{x^2} x^2 + \alpha_{xy} xy + \alpha_{y^2} y^2$$

. Here we are creating a bijection between dual numbers of second order and two variables and polynomials of second order with two variable. We will proceed in this way. We then define $N(n, v)$ to be the number of distinct monomials in v variables up to order n . This can be shown to be equal to

$$N(n, v) = \frac{(n+v)!}{n!v!} = C(n+v, v),$$

Now, assume that these N monomials are arranged in some specific order, indexed by I (a detailed explanation of the ordering used in MAD-NG can be found in the appendix). For each monomial M , we define I_M as its position in this ordering. Conversely, we denote by M_I the I -th monomial in this ordering. For an index I , corresponding to a monomial $M_I = x_1^{i_1} x_2^{i_2} \cdots x_v^{i_v}$, we define $F_I = i_1! i_2! \cdots i_v!$.

Definition 1.3.4 (Algebraic structure of multivariate dual numbers). *We define scalar and vector multiplication in $\mathbb{C}^N$ as follows:*

$$\begin{aligned} (a_1, a_2, \dots, a_N) + (b_1, b_2, \dots, b_N) &= (a_1 + b_1, a_2 + b_2, \dots, a_N + b_N), \\ t \cdot (a_1, a_2, \dots, a_N) &= (t \cdot a_1, t \cdot a_2, \dots, t \cdot a_N), \\ (a_1, a_2, \dots, a_N) \cdot (b_1, b_2, \dots, b_N) &= (c_1, c_2, \dots, c_N), \end{aligned}$$

where the coefficients c_i are defined by:

$$c_i = F_i \sum_{\substack{0 \leq v, \mu \leq N \\ M_v \cdot M_\mu = M_i}} \frac{a_v \cdot b_\mu}{F_v \cdot F_\mu}.$$

Example 2. *To clarify these definitions, let us consider the case case with $n = 2$ and $v = 2$. There are $N = C(2+2, 2) = 6$ monomials in two variables:*

$$1, x, y, x^2, xy, y^2.$$

Using the following ordering and the previous definition of the product, we get:

$$\begin{aligned}
c_1 &= a_1 \cdot b_1, \\
c_2 &= a_1 \cdot b_2 + a_2 \cdot b_1, \\
c_3 &= a_1 \cdot b_3 + a_3 \cdot b_1, \\
c_4 &= 2 \left(\frac{a_1 \cdot b_4}{2} + a_2 \cdot b_2 + \frac{a_4 \cdot b_1}{2} \right), \\
c_5 &= a_1 \cdot b_5 + a_2 \cdot b_3 + a_3 \cdot b_2 + a_5 \cdot b_1, \\
c_6 &= 2 \left(\frac{a_1 \cdot b_6}{2} + a_3 \cdot b_3 + \frac{a_6 \cdot b_1}{2} \right).
\end{aligned}$$

Finally, we define an operator that will be responsible of the derivation (thus our space becomes a differential algebra).

Definition 1.3.5. We introduce the derivative operator ∂_i on the space of vectors. For a vector containing the derivatives of a function f , we define:

$$\partial_v(a_1, a_2, \dots, a_N) = (c_1, c_2, \dots, c_N),$$

where:

$$c_i = \begin{cases} 0 & \text{if } M_i \text{ has order } n, \\ a_{I(M_i \cdot x_v)} & \text{otherwise.} \end{cases}$$

Basically, the operator ∂_v is just shifting the coefficients we have in the vector representing the dual number.

Example 3. Let us suppose we are dealing with this dual number

$$c + \alpha_x \epsilon_x + \alpha_y \epsilon_y + \alpha_{x^2} \epsilon_x^2 + \alpha_{xy} \epsilon_x \epsilon_y + \alpha_{y^2} \epsilon_y^2 = (c, \alpha_x, \alpha_y, \alpha_{xx}, \alpha_{xy}, \alpha_{yy})$$

Then

$$\partial_v(c, \alpha_x, \alpha_y, \alpha_{xx}, \alpha_{xy}, \alpha_{yy}) = (\alpha_x, \alpha_{xx}, \alpha_{xy})$$

With this operation we are now working in a differential algebra (it is easy to prove that the last operation respects the definition of derivative).

1.4 The Importance of Dual Numbers in Accelerator Physics

Dual numbers have become a cornerstone in modern accelerator physics, enabling precise and efficient computations critical for the design and analysis of particle accelerators. Their integration into computational frameworks, particularly for automatic differentiation, has revolutionized approaches to beam dynamics, transfer maps, and perturbative analysis.

In accelerator physics, perturbative methods are frequently employed to study small deviations from reference trajectories or idealized configurations. These methods are vital for understanding how particles respond to imperfections in the accelerator lattice, such as misaligned magnets, energy deviations, or field nonuniformities. Dual numbers naturally align with this framework by encoding these small perturbations into their infinitesimal components, allowing precise calculations of derivatives and sensitivities. This capability eliminates the need for finite-difference methods, which are prone to numerical errors and can become computationally expensive for high-order derivatives. By embedding dual numbers into automatic differentiation tools, perturbative schemes become both more robust and computationally efficient.

Another critical application of dual numbers lies in the computation of transfer maps and Taylor maps, which describe the evolution of particle phase space coordinates through accelerator elements. Traditional methods for generating these maps often involve symbolic manipulation or numerical interpolation, both of which have limitations in terms of scalability and accuracy. By leveraging dual numbers, transfer maps can be computed to arbitrary orders directly, preserving all nonlinear terms with machine precision. This is especially important in modeling nonlinear dynamics, such as the effects of fringe fields, solenoids, and high-order multipoles, where small errors in derivative computation can propagate into significant inaccuracies.

Beam tracking, one of the most computationally intensive tasks in accelerator physics, also benefits greatly from the use of dual numbers. During simulations of particle trajectories through an accelerator lattice, dual numbers enable the simultaneous calculation of a particle's trajectory and its derivatives with respect to various system parameters. This capability is particularly valuable in sensitivity analyses, where small variations in parameters like magnetic field strength or alignment need to be studied systematically. For large-scale projects like the Future Circular Collider (FCC), this dual computation allows researchers to explore vast parameter spaces rapidly, helping to identify optimal configurations for beam stability and luminosity.

Beyond their practical applications, dual numbers play an important role in theoretical investigations within accelerator physics. They are integral to symplectic tracking methods that preserve phase space volume, ensuring consistency with fundamental physical principles. Moreover, their compatibility with higher-order dynamics and differential algebra frameworks allows for advanced studies of resonance phenomena and intricate phase space structures.

The integration of dual numbers into software tools like MAD-NG exemplifies their transformative impact on accelerator physics. By embedding dual-number-based automatic differentiation into these frameworks, researchers gain access to unprecedented levels of precision and computational speed. This not only accelerates the development of next-generation accelerator designs but also deepens our understanding of the nonlinear and perturbative dynamics that govern beam behavior.

In summary, dual numbers are indispensable in modern accelerator physics, enabling precise derivative computation and enhancing the efficiency of perturbative analyses, transfer map generation, and beam tracking. Their adoption in computational tools ensures they will remain a cornerstone of accelerator design and optimization in the years to come.

1.4.1 The Computation of Transfer Maps

Differential Algebras provide an efficient tool for computing the transfer map in accelerator physics.

To demonstrate this approach, let us consider a simple example of midplane motion in a 90° homogeneous bending magnet. The initial displacement is denoted by x_i , and the scaled transverse momentum relative to the reference trajectory is defined as $a_i = \sin(x'_1)$ (see Fig. 1.1). We are interested in calculating the final values, x_f and $a_f = \sin(x'_2)$. Since the particle trajectories inside the magnet follow circular arcs, we can directly apply the geometry shown in Fig. 1.1 to derive the following equations:

$$A = R \sin(x'_1) = R a_i, \quad (1.1)$$

$$B = R(1 - \cos(x'_1)) + x_i = R \left(1 - \sqrt{1 - a_i^2}\right) + x_i, \quad (1.2)$$

$$a_f = \sin(x'_2) = -\frac{B}{R}, \quad (1.3)$$

$$x_f = A - R(1 - \cos x'_2) = A - R \left(1 - \sqrt{1 - a_f^2}\right). \quad (1.4)$$

These equations allow for the computation of the final position x_f and transverse momentum a_f in terms of the initial values. By applying differential algebra to these expressions, we can also compute the derivatives of x_f and a_f with respect to x_i and a_i . Evaluating these derivatives at $x_i = 0$ and $a_i = 0$ gives us the expansion coefficients for the transfer map of a bending magnet.

For clarity, let us explicitly show how x_f and a_f are computed using differential algebra. By following the previous algebraic rules identifying the variable x with the monomial x , and a with y , we obtain the following:

$$\begin{aligned}
x_i &= (0, 1, 0, 0, 0, 0), \\
a_i &= (0, 0, 1, 0, 0, 0), \\
A &= (0, 0, R, 0, 0, 0), \\
B &= (0, 1, 0, 0, 0, R), \\
a_f &= \left(0, -\frac{1}{R}, 0, 0, 0, -1\right), \\
x_f &= \left(0, 0, R, -\frac{1}{R}, 0, 0\right).
\end{aligned}$$

By comparing these results with a matrix-based approach, we find complete agreement. For example, the fact that the second component of x_f is zero implies that $\frac{\partial x_f}{\partial x_i} = 0$, which is a well-known property of 90° bending magnets, where the transport matrix element $R_{1,1} = 0$.

If additional particle optical elements follow this bending magnet, one need not start the calculations from $x_i = (0, 1, 0, 0, 0, 0)$ and $a_i = (0, 0, 1, 0, 0, 0)$. Instead, one can directly use the previously computed values of x_f and a_f from Eq. (39) as the starting point. This eliminates the need for the often complex concatenation process, resulting in significant performance improvement.

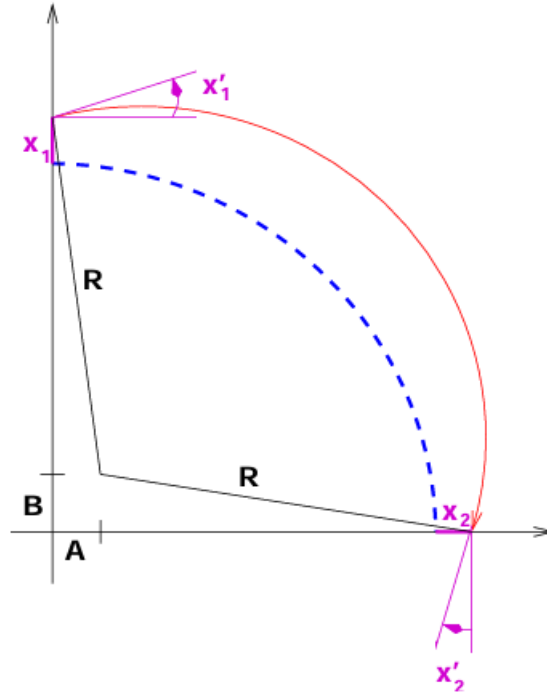


Figure 1.1: Geometry of the particle trajectory in a 90° bending magnet.

1.4.2 Numerical maps approach

For most beam physics systems, deriving analytical formulas for transfer maps is not feasible, except in rare special cases. Nevertheless, it is possible to compute the relationship between final and initial coordinates numerically, using integration techniques. Essentially, a numerical integration algorithm represents a complex function composed of a finite sequence of elementary operations and functions. Due to its inherent complexity, expressing this function analytically—or differentiating it to high orders with respect to phase-space coordinates or system parameters—is exceedingly challenging.

The differential algebraic (DA) method offers a powerful solution to this problem by enabling the computation of high-order maps in a straightforward manner. This approach involves replacing every elementary operation and function within the numerical algorithm with their respective counterparts in differential algebra. One of the key advantages of this method is that the computational effort for evaluating derivatives is largely independent of the function's complexity.

1.4.3 Real value of TPSA in Accelerator physics

After this brief discussion about the use of Truncated Power Series Algebra we can understand that the power of this method lies in the underlying equivalence relation between functions in the 'normal world' and functions in the 'dual number world'. Indeed we need to think about the fact that in all the common functions we use are not a gift of God but a gift of differential equations. If we need to solve this equations

$$y' = y$$

we know that the solution is e^x and we call it the exponential function. So we understand that the 'exponential function' and e^x are just a name and a symbol we use to describe the solution of that differential equation. In fact, are you able to calculate with pen and paper the value e^1 ? And now that you noticed that you do not know how to compute it you should think about the fact that our CPUs have the same problem. But if I provide you

$$e^x = \sum_{i=0}^{\infty} \frac{x^i}{i!}$$

now you (and the CPU) are able to compute at least an approximated (as precise as you want) value of e^1 with pen and paper. This stupid example is very useful to understand the importance of Power Series: they are not only a computational tool but also a cognitive tool. That being said, we comprehend that we can represent our common analytical functions as power series (again, the name of the function is just a symbol) and we can build this formal relation with power series

$$f(x) \Leftrightarrow \sum_{i=0}^{\infty} \mathcal{C}_i x^i$$

. Specifically, if we use Taylor Series we get

$$f(x) \Leftrightarrow \sum_{i=0}^{\infty} \frac{f^{(i)}(0)}{i!} x^i$$

. Now, if we move from the real world to the 'dual number world' we extend this approach as we have already presented

$$f((a, \epsilon)) := f(a + \epsilon) \Leftrightarrow \sum_{i=0}^{\infty} \frac{f^{(i)}(a)}{i!} \epsilon^i$$

if we then take a differential unit with an order of nilpotency = n the previous formula will be truncated

$$f((a, \epsilon)) \Leftrightarrow \sum_{i=0}^n \frac{f^{(i)}(a)}{i!} \epsilon^i$$

Thus we see that if before for each analytical function we have an 'equivalence' with an infinite power series, now we have an equivalence with a truncated power series. To stress more the concept, in the 'real' world each function is a an infinite summation of monomials whereas in the 'dual' world the same function will be a finite summation of monomials. So in our new world we work directly with truncated series because this is the representation of this functions in this weird world. And now that we have this formalism we can see the power of the TPSA approach. Indeed in transverse dynamics we are often studying the behavior of the beam with a perturbative approach with respect to the closed orbit; thus we need to know how the phase space is being deformed under the action of the various orders of the map we are using. To give a more concrete example: let $\mathcal{M}(z)$ be a 1-dimensional map and suppose that z_f is a fixed point of the map, i.e. $z_f = \mathcal{M}(z_f)$. Moreover let us suppose that the map $\mathcal{M}$ is a 'simple' composition of common functions

$$\mathcal{M}(z) = \sinh(\tanh(\arccos(\text{Faddeeva}(\frac{1}{z})))) = \sinh\left(\tanh\left(\arccos\left(e^{-\frac{1}{z^2}}\left(1 + \frac{2i}{\sqrt{\pi}} \int_0^{1/z} e^{t^2} dt\right)\right)\right)\right)$$

and we want to study how the map is deforming this 1 dimensional phase space around the fixed point. We immediately see that even getting the first order (derivative) correction of the map is a computational nightmare and we face numerical instabilities for the first order already with finite differences and Mathematica takes about 2 seconds to compute the derivative symbolically (and still it needs to compute the numerical value). Now, if we move to the Truncated Power Series Algebra, everything will be very easy. Again we stress the idea that common functions are just summation of polynomials and we define

$$T_f((z, \epsilon)) = \sum_{i=0}^n \frac{f^{(i)}(z)}{i!} \epsilon^i$$

. If we want to study only a linearized system then each function will be represented with a differential unit whose order of nilpotency is equal to 2 ($f(a, \epsilon) = f(a) + f'(a)\epsilon$). This then implies that

$$\mathcal{M}((z, \epsilon)) = T_{\sinh}(T_{\tanh}(T_{\arccos}(T_{\text{Faddeeva}}(T_{\text{Inv}}((z, \epsilon)))))) = \mathcal{M}(z) + \mathcal{M}'(z)\epsilon$$

automatically. Basically, the monstrous problem has become just a composition of polynomials inside the dual number algebra. If now we stop playing with the previous map and we move to a real And yet, as ugly as this function is, this is nothing compared to the thousands of lines of code and all the maps applied for a lattice in accelerator physics. That is why starting directly with an object belonging to the TPSA will track automatically as many perturbative terms as we need: all this terms then will be used to understand how the phase space is deformed through the action of the map around the closed orbit. Naturally we still need to pay 'a cost'(nothing is given fo free): the cost in this case relies in the fact that we need a concrete and stable truncated power series representation of our functions. After we have such a representation, we can study all our favorite systems keeping trace of as many perturbative terms as we want.

1.5 Applications of Dual Numbers Beyond Accelerator Physics

While dual numbers have become a cornerstone in accelerator physics, their utility extends to a wide array of other fields. Their ability to perform exact and efficient derivative computations makes them an invaluable tool across various domains in science, engineering, and optimization. Furthermore, their potential applications in emerging areas suggest that dual numbers may continue to grow in importance in the future.

One of the most prominent uses of dual numbers is in the field of optimization, particularly in machine learning and artificial intelligence. Neural network training often relies on gradient-based optimization techniques, such as stochastic gradient descent (SGD). Traditionally, these gradients are computed using backpropagation, which can be both computationally expensive and prone to numerical instabilities in certain scenarios. Dual numbers provide a direct and numerically stable approach for calculating gradients, bypassing the need for symbolic differentiation or finite differences. Their adoption in automatic differentiation libraries like JAX and TensorFlow has already begun to enhance the efficiency of deep learning frameworks. Future advancements in AI, particularly in the development of more interpretable and robust models, could further leverage the precision and flexibility of dual numbers.

In computer graphics and computational geometry, dual numbers are employed for tasks such as ray tracing, collision detection, and the computation of normals in 3D models. By embedding derivative computations into geometric algorithms, dual numbers allow for smoother and more accurate simulations of physical interactions, such as light propagation or object deformation. This capability is especially useful in fields like gaming, virtual reality, and computer-aided design (CAD), where real-time rendering and precision are crucial.

Robotics and control systems also benefit significantly from dual numbers. In these fields, systems are often governed by complex equations of motion, where derivatives with respect to state variables or control inputs are critical for stability and optimization. Dual numbers provide a systematic way

to compute Jacobians and Hessians for such systems, enabling more efficient trajectory planning, parameter tuning, and real-time control. As robotics evolves toward more autonomous and adaptive systems, dual numbers could play a key role in facilitating on-the-fly adjustments and optimizations.

Another potential application lies in scientific computing and numerical simulations, particularly in fluid dynamics and structural analysis. In these domains, partial differential equations (PDEs) are frequently solved to model physical phenomena such as airflow, heat transfer, or stress distribution. Dual numbers can be incorporated into numerical solvers to compute sensitivities and gradients of these PDEs with respect to boundary conditions or material properties. This approach not only enhances the accuracy of simulations but also enables efficient design optimization, making dual numbers a valuable addition to computational engineering workflows.

In addition to their established applications, dual numbers hold promise for emerging areas of research. In quantum computing, for instance, their ability to encode and propagate derivative information could facilitate the development of hybrid quantum-classical algorithms, particularly for tasks like variational quantum eigensolvers (VQEs) and quantum machine learning. Similarly, in the burgeoning field of computational biology, dual numbers could be used to model biochemical networks or optimize drug design through the precise computation of sensitivities and reaction rates.

Finally, dual numbers have potential applications in finance and economics. Derivative calculations are central to modeling complex financial instruments, assessing risk, and optimizing investment strategies. By integrating dual numbers into quantitative finance tools, one could achieve faster and more accurate computations of key metrics such as sensitivities (the so-called "Greeks") in option pricing or portfolio optimization.

In summary, the applications of dual numbers extend far beyond their origins in mathematical analysis and accelerator physics. From machine learning to robotics, and from scientific computing to emerging fields like quantum computing and computational biology, dual numbers have proven to be a versatile and powerful tool. Their ability to combine precision with computational efficiency ensures they will remain a valuable asset in addressing the challenges of an increasingly complex and interdisciplinary world.

Chapter 2

Enhancing the TPSA library in MAD-NG

2.1 Motivation

In this chapter we are presenting new analytical series for some common functions. The need for this improvements relies on the fact that computing directly these functions within the Truncated Power Series Algebra will generate important numerical instabilities, especially for large orders. As an example if we evaluate

$$\arctan(x) = \tan^{-1}(x) = \left(\frac{\sin}{\cos} \right)^{-1}(x)$$

directly in the Truncated Power Series Algebra, relying on the primitive functions $\sin$, $\cos$ and the inversion, which are already very stable, we will face large instabilities that are growing with the orders. For this reason we had to find a new way to evaluate those functions. Specifically, $\arctan$, $\arctanh$, arccot , $\operatorname{arccoth}$, and $\operatorname{arccosh}$ were presenting instabilities for high orders (> 6) whereas for the other functions the instabilities were also present for lower orders.

2.2 Atan, Acot, Atanh, Acoth

Here we are going to present and prove some new analytical series for the functions atan , acot , atanh , acoth . We put all this functions together because their series and the derivations are really similar. Let us start first with some definitions.

Definition 2.2.1. *We define the n -th hypertriangular number in d dimension as:*

$$\mathcal{T}^d(n) := \begin{cases} 1 & \text{if } d = 0 \\ \frac{(n) \times (n+1) \times \dots \times (n+d-1)}{(d)!} & \text{if } d \geq 1 \end{cases}$$

Order	Derivative
0	$\arctan(x)$
1	$\frac{1}{x^2+1}$
2	$-\frac{2x}{(x^2+1)^2}$
3	$\frac{2\left(\frac{4x^2}{x^2+1}-1\right)}{(x^2+1)^2}$
4	$\frac{24x\left(-\frac{2x^2}{x^2+1}+1\right)}{(x^2+1)^3}$
5	$\frac{24\left(\frac{16x^4}{(x^2+1)^2}-\frac{12x^2}{x^2+1}+1\right)}{(x^2+1)^3}$
6	$\frac{240x\left(-\frac{16x^4}{(x^2+1)^2}+\frac{16x^2}{x^2+1}-3\right)}{(x^2+1)^4}$
7	$\frac{720\left(\frac{64x^6}{(x^2+1)^3}-\frac{80x^4}{(x^2+1)^2}+\frac{24x^2}{x^2+1}-1\right)}{(x^2+1)^4}$
8	$\frac{40320x\left(-\frac{16x^6}{(x^2+1)^3}+\frac{24x^4}{(x^2+1)^2}-\frac{10x^2}{x^2+1}+1\right)}{(x^2+1)^5}$
9	$\frac{40320\left(\frac{256x^8}{(x^2+1)^4}-\frac{448x^6}{(x^2+1)^3}+\frac{240x^4}{(x^2+1)^2}-\frac{40x^2}{x^2+1}+1\right)}{(x^2+1)^5}$

Table 2.1: Derivatives of $\arctan(x)$ up to 13th order

There are a lot of interesting relations for this hypertriangular numbers. I proved some of them but I could further investigate their properties. For instance, here I provide the simplest property that will be used later.

Lemma 2.2.2. *The hypertriangular numbers can be alternatively defined as*

$$\mathcal{T}^d(n) := \binom{n+d-1}{d}$$

Proof. Simple calculation. □

We define the double factorial as it will be used

Definition 2.2.3 (Double Factorial). *The double factorial of a non-negative integer n , denoted by $n!!$, is defined as the product of all the integers from n down to 1 that have the same parity (odd or even) as n . Formally:*

$$n!! = \begin{cases} n \times (n-2) \times (n-4) \times \cdots \times 4 \times 2, & \text{if } n \text{ is even,} \\ n \times (n-2) \times (n-4) \times \cdots \times 3 \times 1, & \text{if } n \text{ is odd.} \end{cases}$$

Additionally, by convention:

$$0!! = 1 \quad \text{and} \quad (-1)!! = 1.$$

Now we introduce another auxiliary function to have a more compact formula

$$f(n) = (n-1) \times ((n+1) \bmod 2) + 1$$

Thus we can finally write the generic derivative of $\arctan(x)$.

Theorem 2.2.4.

$$\arctan^{(n)}(x) = \frac{(n-1)! \left[x^{((n+1) \bmod 2)} (-1)^{n+\lfloor \frac{n-3}{2} \rfloor} f(n) + \sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor} (-1)^{n+i+1} 2^{n-1-2i} \mathcal{J}^i(n-2i) \frac{x^{n-1-2i}}{(x^2+1)^{\lfloor \frac{n-1-2i}{2} \rfloor}} \right]}{(x^2+1)^{\lceil \frac{n+1}{2} \rceil}} \quad (2.1)$$

Proof. To prove the previous relation we will follow the easy path of induction. At first we observe that this relation obviously holds for the previously showed cases in the table of derivatives. Then we are gonna prove by induction the relation dividing the cases in odd and even integers. In this proof we will use 2.1 as

$$\alpha_i^n := (-1)^{n+i+1} 2^{n-1-2i} \mathcal{J}^i(n-2i)$$

We start with an odd n .

Case 1: n odd

In this case we can rewrite 2.1 as

$$\arctan^{(n)}(x) = (n-1)! \left[\frac{(-1)^{n+\frac{n-3}{2}}}{(x^2+1)^{\frac{n+1}{2}}} + \sum_{i=0}^{\frac{n-3}{2}} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right]$$

We then focus on the second term to obtain an interesting relation. Indeed deriving we obtain

$$\frac{\partial}{\partial x} \left(\sum_{i=0}^{\frac{n-3}{2}} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right) = \sum_{i=0}^{\frac{n-3}{2}} \alpha_i^n \left(\frac{2x^{n-2i}(i-n)}{(x^2+1)^{n-i+1}} + \frac{x^{n-2i-2}(n-2i-1)}{(x^2+1)^{n-i}} \right)$$

From this we see that, summing the monomials with the same orders, we get

$$\frac{x^{n-2i-2}(n-2i-1)}{(x^2+1)^{n-i}} ((n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n)$$

Thus we need to check that

$$(n-1)!(n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n = (n!)\alpha_{i+1}^{n+1}$$

To have a better expression for the left-hand side we compute

$$\frac{\alpha_i^n}{\alpha_{i+1}^n} = -4 \frac{(i+1)(n-i-1)}{(n-2i-1)(n-2i-2)}$$

Thus the left-hand side reads

$$(n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n = \alpha_{i+1}^n \left(-4 \frac{(i+1)(n-i-1)}{(n-2i-2)} + 2(i+1-n) \right)$$

Writing now explicitly α_{i+1}^n we get

$$(n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n = \frac{-2n(n-i-1)}{n-2i-2} (-1)^{n+i+2} 2^{n-2i-3} \mathcal{F}^{i+1}(n-2i-2)$$

Now writing explicitly also α_{i+1}^{n+1}

$$\alpha_{i+1}^{n+1} = (-1)^{n+i+3} 2^{n-2i-2} \mathcal{F}^{i+1}(n-2i-1)$$

we see that we only need to check

$$\frac{\mathcal{F}^{i+1}(n-2i-2) \frac{-2n(n-i-1)}{n-2i-2}}{n \mathcal{F}^{i+1}(n-2i-1)} = 1$$

and, using the definition of the hypertriangular number and their expression through binomial coefficient, this is easily demonstrable.

Now we are left with proving the corner cases, i.e. $i = 0$ and $i = \frac{n-3}{2}$. In the first case we easily see

$$(n-1)! \alpha_i^n \frac{2x^{n-2i}(i-n)}{(x^2+1)^{n-i+1}} \Big|_{i=0} = -(n!) \frac{2x^n}{(x^2+1)^{n+1}} = -(n!) \alpha_0^{n+1} \frac{2x^n}{(x^2+1)^{n+1}}$$

that is we have an equality of coefficients for this fraction of monomials. For the case $i = \frac{n-1}{3}$ we will need to take into account the derivative of the piece outside the summation of 2.1 $\frac{(-1)^{n+\frac{n-3}{2}}}{(x^2+1)^{\frac{n+1}{2}}}$ and sum it with the contribution coming from the summation. In the end we get

$$(n-1)! \frac{x}{x(n+1)} (x^2+1)^{\frac{n+3}{2}} \left(2\alpha_{\frac{n-3}{2}}^n - (n+1) \right)$$

and we need to check that this is equal to the part outside the summation of 2.1 computed with $n+1$. Using the previous properties this is easily checked.

Case 2: n even

Now we start to prove the case with n even. In this case the formula 2.1 reads

$$\arctan^{(n)}(x) = (n-1)! \left[\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}} + \sum_{i=0}^{\frac{n-3}{2}-\frac{1}{2}} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right]$$

. In this case we immediately see that we need to check only the corner cases since the relation between

α_i^n , α_{i+1}^n and α_{i+1}^{n+1} is still valid. The case $i = 0$ is as before. We now need to check $\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}}$. Derivating this part of the equation will give a monomial with a numerator of 0 degree and another monomial that will be summed with the contribution coming from the summation. The first term is easily checked since we just need to compute it and compare it with the same monomial of the equation for the derivative of order $n + 1$. We thus get

$$\frac{\partial}{\partial x} \left(\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}} \right) = \frac{(-1)^{\frac{3n-4}{2}} n}{(x^2+1)^{\frac{n}{2}+1}} + \frac{(-1)^{\frac{3n-4}{2}} 2nx^2(-\frac{n}{2}-1)}{(x^2+1)^{\frac{n}{2}+2}}$$

One can easily check that computing the term of the same order of $\frac{(-1)^{\frac{3n-4}{2}}(n!)}{(x^2+1)^{\frac{n}{2}+1}}$ for the formula of degree $n + 1$ will have the same coefficient. We thus miss the last piece. To prove that the second piece obtained through the derivation of $\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}}$ summed with the coefficient obtained through the derivation of the summation part will yield the correct coefficient α_{i+1}^{n+1} . To prove this we notice that

$$n(-1)^{\frac{3n}{2}} = \alpha_{\frac{n-3}{2}+1}^n$$

. So, since we know that the relation $(n-1)!((n-2i-1)\alpha_i n + 2(i+1-n)\alpha_{i+1}^n) = (n!)\alpha_{i+1}^{n+1}$, we just need to check that in our case

$$(i+1-n) = -(1 + \frac{n}{2})$$

and substituting $i = \frac{n-3}{2} + \frac{1}{2}$ we achieve the result we wanted. $\square$

Now we will prove that formula 2.1 can be rewritten in a simpler way.

Corollary 2.2.5. *Defining the coefficients*

$$\alpha_i^n := (-1)^{n+i+1} 2^{n-1-2i} \mathcal{J}^i(n-2i)$$

formula 2.1 can be rewritten as

$$\arctan^{(n)}(x) = (n-1)! \left(\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right) \quad (2.2)$$

Proof. The proof is easy indeed if we compute

$$\lfloor \frac{n-1-2i}{2} \rfloor + \lceil \frac{n+1}{2} \rceil = n-i$$

. Moreover we now need to prove that $x^{((n+1) \bmod 2)} (-1)^{n+\lfloor \frac{n-3}{2} \rfloor} f(n)$ in 2.1 can be written as $\alpha_{\lfloor \frac{n-3}{2} \rfloor + 1}$.

When n is even we already know that this is true since we used it in the proof of 2.2.4. We then need just to prove it when n is odd. In this case we see

$$x^{((n+1) \bmod 2)} (-1)^{n + \lfloor \frac{n-3}{2} \rfloor} f(n) = (-1)^{\frac{3n-3}{2}}$$

and

$$\alpha_{\lfloor \frac{n-3}{2} \rfloor + 1}^n = \quad (2.3)$$

$$= \alpha_{\frac{n-1}{2}}^n \quad (2.4)$$

$$= (-1)^{\frac{3n+1}{2}} 2^0 \mathcal{T}^{\frac{n-1}{2}}(1) \quad (2.5)$$

$$= (-1)^{\frac{3n-3}{2}} \times 1 \quad (2.6)$$

The last piece to be checked is that the numerator does not have any power of x . This is easily checked

$$x^{n-1-2i} \big|_{i=\frac{n-1}{2}} = x^0$$

For completeness we write the proof also for the case of n even. In this case we have

$$x^{((n+1) \bmod 2)} (-1)^{n + \lfloor \frac{n-3}{2} \rfloor} f(n) = x(-1)^{\frac{3n-4}{2}} n$$

and

$$\alpha_{\lfloor \frac{n-3}{2} \rfloor + 1}^n = \quad (2.7)$$

$$= \alpha_{\frac{n-2}{2}}^n \quad (2.8)$$

$$= (-1)^{\frac{3n}{2}} 2^1 \mathcal{T}^{\frac{n-2}{2}}(2) \quad (2.9)$$

$$= (-1)^{\frac{3n-4}{2}} 2 \left(\frac{\frac{n}{2}}{\frac{n-2}{2}} \right) \quad (2.10)$$

$$= (-1)^{\frac{3n-4}{2}} 2 \frac{n}{2} \quad (2.11)$$

$$= (-1)^{\frac{3n-4}{2}} n \quad (2.12)$$

And in the end we check that

$$x^{n-2i-1} \big|_{i=\frac{n-2}{2}} = x^1$$

□

Thus we have some interesting results:

Corollary 2.2.6. *The Taylor expansion of $atan$ around x reads:*

$$\mathcal{T}_{atan(x)}^n(y) = atan(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i+1} 2^{n-1-2i} \mathcal{T}^i(n-2i) x^{n-1-2i} \right]}{n(x^2+1)^{n-i}} y^n \quad (2.13)$$

Proof. Obvious. □

In a trivial way one can get the expansion of $arccot$ around x in the same way, obtaining (only a minus sign difference):

Theorem 2.2.7. *The Taylor expansion of $acot$ around x reads:*

$$\mathcal{T}_{acot(x)}^n(y) = acot(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i} 2^{n-1-2i} \mathcal{T}^i(n-2i) x^{n-1-2i} \right]}{n(x^2+1)^{n-i}} y^n \quad (2.14)$$

Proof. Same proof as theorem 2.2.4 □

One can extend these formulas also to the hyperbolic counterparts to obtain:

Theorem 2.2.8. *The Taylor expansion of $atanh$ around x reads:*

$$\mathcal{T}_{atanh(x)}^n(y) = atanh(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i} 2^{n-1-2i} \binom{n-i-1}{i} x^{n-1-2i} \right]}{n(x^2-1)^{n-i}} y^n \quad (2.15)$$

Proof. Same proof as theorem 2.2.4 □

Theorem 2.2.9. *The Taylor expansion of $acoth$ around x reads:*

$$\mathcal{T}_{acoth(x)}^n(y) = acoth(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i} 2^{n-1-2i} \binom{n-i-1}{i} x^{n-1-2i} \right]}{n(x^2-1)^{n-i}} y^n \quad (2.16)$$

Proof. Same proof as theorem 2.2.4 □

Another way to find the formulas is to use the complex formulation of the previous functions. Let take atan again as an example. We then have that:

$$\begin{aligned}
\frac{d^n}{dx^n} \arctan(x) &= \frac{d^n}{dx^n} \frac{i}{2} \log\left(\frac{x-i}{x+i}\right) \\
&= \frac{1}{2i} \frac{d^{n-1}}{dx^{n-1}} \left(\frac{1}{x-i} - \frac{1}{x+i} \right) \\
&= \frac{(-1)^{n-1}(n-1)!}{2i} \left(\frac{1}{(x-i)^n} - \frac{1}{(x+i)^n} \right) \\
&= \frac{(-1)^{n-1}(n-1)!}{2i} \cdot \frac{(x+i)^n - (x-i)^n}{(x^2+1)^n} \\
&= \frac{(-1)^{n-1}(n-1)!}{2i} \cdot \frac{\sum_{k=0}^n \binom{n}{k} x^{n-k} (i^k - (-i)^k)}{(x^2+1)^n} \\
&= \frac{(n-1)!}{(x^2+1)^n} \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n}{2k+1} x^{n-2k-1} (-1)^{n+k-1} \quad (1, 3)
\end{aligned}$$

This formula seems really similar to the previous one but actually we have a substantial difference: we notice that here we are summing powers of x without dividing them by powers of (x^2+1) . Also the coefficient multiplying the powers of x is different. Thanks to these two different expressions we can write a new formula for the binomial coefficient:

$$\sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \left(\binom{n}{2k+1} - 2^{n-1-2k} \binom{n-i-k}{i} (x^2+1)^k \right) \frac{x^{n-2k-1}}{(x^2+1)^n} = 0$$

Expanding now $(x^2+1)^k$ we find that $\forall k$ and $\forall n$ (with $k \leq \lfloor \frac{n-1}{2} \rfloor$):

$$\binom{n}{2k+1} = \sum_{i=0}^{\lfloor \frac{n-1}{2} \rfloor - k} (-1)^i 2^{n-1-2(k+i)} \binom{n-(k+i)-1}{k+i} \binom{k+i}{k}$$

This is indeed a formula (we do not know if it is new) for the binomial coefficients for odd numbers. We can follow the exact same procedure for $\arctan$. In this case we get :

$$\begin{aligned}
\frac{d^n}{dx^n} \operatorname{acot}(x) &= \frac{d^n}{dx^n} \frac{i}{2} \log\left(\frac{x-i}{x+i}\right) \\
&= -\frac{d^n}{dx^n} \frac{1}{2i} \log\left(\frac{x-i}{x+i}\right) \\
&\quad \vdots \\
&= \frac{(n-1)!}{(x^2+1)^n} \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n}{2k+1} x^{n-2k-1} (-1)^{n+k} \quad (1,3)
\end{aligned}$$

We then proceed to obtain the other 2 alternative expansions for atanh and acoth :

$$\begin{aligned}
\frac{d^n}{dx^n} \operatorname{atanh}(x) &= \frac{d^n}{dx^n} \frac{1}{2} \log\left(\frac{1+x}{1-x}\right) \quad \text{if } |x| < 1 \\
&= \frac{1}{2} \frac{d^{n-1}}{dx^{n-1}} \left(\frac{1}{1+x} + \frac{1}{1-x} \right) \\
&= \frac{(-1)^{n-1} (n-1)!}{2i} \left(\frac{1}{(1+x)^n} - \frac{1}{(x-1)^n} \right) \\
&= \frac{(-1)^{n-1} (n-1)!}{2} \cdot \frac{(x-1)^n - (x+1)^n}{(x^2-1)^n} \\
&= \frac{(-1)^{n-1} (n-1)!}{2} \cdot \frac{\sum_{k=0}^n \binom{n}{k} (x)^{n-k} ((-1)^k - 1)}{(x^2-1)^n} \\
&= \frac{(n-1)!}{(x^2-1)^n} \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n}{2k+1} x^{n-2k-1} (-1)^{n+k} \quad (1,3)
\end{aligned}$$

and

$$\begin{aligned}
\frac{d^n}{dx^n} \operatorname{acoth}(x) &= \frac{d^n}{dx^n} \frac{1}{2} \log\left(\frac{1+x}{1-x}\right) \quad \text{if } |x| > 1 \\
&\quad \vdots \\
&= \frac{(n-1)!}{(x^2-1)^n} \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n}{2k+1} x^{n-2k-1} (-1)^{n+k} \quad (1,3)
\end{aligned}$$

2.3 Acosh

We now analyze $\text{acosh}(x)$ to find another series to be applied in our differential algebra of truncated series.

Order	Derivative
0	$\text{acosh}(x)$
1	$\frac{1}{\sqrt{(x-1)(x+1)}}$
2	$-\frac{\frac{1}{x+1} + \frac{1}{x-1}}{2\sqrt{(x-1)(x+1)}}$
3	$\frac{\frac{3}{(x+1)^2} + \frac{2}{(x-1)(x+1)} + \frac{3}{(x-1)^2}}{4\sqrt{(x-1)(x+1)}}$
4	$-\frac{3\left(\frac{5}{(x+1)^3} + \frac{3}{(x-1)(x+1)^2} + \frac{3}{(x-1)^2(x+1)} + \frac{5}{(x-1)^3}\right)}{8\sqrt{(x-1)(x+1)}}$
5	$\frac{3\left(\frac{35}{(x+1)^4} + \frac{20}{(x-1)(x+1)^3} + \frac{18}{(x-1)^2(x+1)^2} + \frac{20}{(x-1)^3(x+1)} + \frac{35}{(x-1)^4}\right)}{16\sqrt{(x-1)(x+1)}}$
6	$-\frac{15\left(\frac{63}{(x+1)^5} + \frac{35}{(x-1)(x+1)^4} + \frac{30}{(x-1)^2(x+1)^3} + \frac{30}{(x-1)^3(x+1)^2} + \frac{35}{(x-1)^4(x+1)} + \frac{63}{(x-1)^5}\right)}{32\sqrt{(x-1)(x+1)}}$
7	$\frac{45\left(\frac{231}{(x+1)^6} + \frac{126}{(x-1)(x+1)^5} + \frac{105}{(x-1)^2(x+1)^4} + \frac{100}{(x-1)^3(x+1)^3} + \frac{105}{(x-1)^4(x+1)^2} + \frac{126}{(x-1)^5(x+1)} + \frac{231}{(x-1)^6}\right)}{64\sqrt{(x-1)(x+1)}}$
8	$-\frac{315\left(\frac{429}{(x+1)^7} + \frac{231}{(x-1)(x+1)^6} + \frac{189}{(x-1)^2(x+1)^5} + \frac{175}{(x-1)^3(x+1)^4} + \frac{175}{(x-1)^4(x+1)^3} + \frac{189}{(x-1)^5(x+1)^2} + \frac{231}{(x-1)^6(x+1)} + \frac{429}{(x-1)^7}\right)}{128\sqrt{(x-1)(x+1)}}$
9	$\frac{315\left(\frac{6435}{(x+1)^8} + \frac{3432}{(x-1)(x+1)^7} + \frac{2772}{(x-1)^2(x+1)^6} + \frac{2520}{(x-1)^3(x+1)^5} + \frac{2450}{(x-1)^4(x+1)^4} + \frac{2520}{(x-1)^5(x+1)^3} + \frac{2772}{(x-1)^6(x+1)^2} + \frac{3432}{(x-1)^7(x+1)} + \frac{6435}{(x-1)^8}\right)}{256\sqrt{(x-1)(x+1)}}$

Table 2.2: Derivatives of $\text{arcosh}(x)$ up to 9th order

In the end the analytical generic formula for derivations of $\text{arcosh}(x)$ reads:

Theorem 2.3.1.

$$\text{arcosh}^{(n)}(x) = \frac{(-1)^{(n+1)}}{2^{(n-1)}(x^2-1)^{\frac{1}{2}}} \sum_{i=0}^{\lfloor \frac{(n-1)}{2} \rfloor} \frac{(2n-3-2i)!! \mathcal{T}^i(n-i)(2i-1)!!}{2^{\delta(n-2i-1)}} \left(\frac{1}{(x+1)^{n-i-1}(x-1)^i} + \frac{1}{(x+1)^i(x-1)^{n-i-1}} \right) \quad (2.17)$$

where $\delta(x)$ is the Kronecker delta.

Proof. As before we define

$$\alpha_i^n := (2n-3-2i)!! \mathcal{T}^i(n-i)(2i-1)!!$$

Then we proceed as before by induction. First we can trivially see that the formula is correct for a starting n_0 case. Then we will prove that if the formula is valid for the n -th order derivative, it is also valid for the $(n+1)$ -th order derivative.

Computing the derivative of the formula above we see indeed that

$$\begin{aligned}
& \frac{\partial}{\partial x} \left(\frac{1}{(x+1)^{n-i-\frac{1}{2}}(x-1)^{i+\frac{1}{2}}} + \frac{1}{(x+1)^{i+\frac{1}{2}}(x-1)^{n-i-\frac{1}{2}}} \right) \\
&= \left(\frac{1}{2} + i - n \right) \left(\frac{1}{(x-1)^{n-i+\frac{1}{2}}(x+1)^{i+\frac{1}{2}}} + \frac{1}{(x+1)^{n-i+\frac{1}{2}}(x-1)^{i+\frac{1}{2}}} \right) \\
&+ \left(-i - \frac{1}{2} \right) \left(\frac{1}{(x-1)^{n-i-\frac{1}{2}}(x+1)^{i+\frac{3}{2}}} + \frac{1}{(x+1)^{n-i-\frac{1}{2}}(x-1)^{i+\frac{3}{2}}} \right)
\end{aligned}$$

From this we notice that the relation we need to check is

$$\alpha_{i+1}^{n+1} = \left(-\frac{1}{2} - i \right) \alpha_i^n + \left(\frac{3}{2} + i - n \right) \alpha_{i+1}^n$$

This is long and tedious but easy and one can proceed as in the proof of atan □

2.4 Faddeeva function

The Faddeeva function, denoted as $f(z)$, is a complex function widely used in physics and engineering, particularly in problems involving wave propagation and scattering in complex media. In the case of accelerator physics it is used when dealing with the 'beam-beam' effect: this phenomenon is a nonlinear interaction in particle colliders caused by the electromagnetic fields of colliding charged particle beams. These fields act as focusing lenses, perturbing particle trajectories and altering their oscillation frequencies (tunes). This can lead to tune shifts, resonance-driven instabilities, and phase-space distortions, affecting beam quality and stability. Managing this effect is critical in high-luminosity colliders to ensure efficient operation and maximize collision rates. Now, coming back to the Faddeeva function, it is defined as:

$$w(z) = e^{-z^2} \left(1 + \frac{2i}{\sqrt{\pi}} \int_0^z e^{t^2} dt \right),$$

where z is a complex variable. This function includes an exponential decay term, e^{-z^2} , and an integral that introduces a complex component.

For the Faddeeva function there is a very well known recursive relation to express and evaluate the derivatives of the function

$$w^{(n+2)} = -2zw^{(n+1)} - 2(n+1)w^{(n)} \quad n = 0, 1, \dots$$

The problem of this recursive relation lies in the fact that it is very unstable (already at $z = 5$ we lose a lot of accuracy and many orders are wrong; if we further increase z , we will see a larger and larger discrepancy, i.e. values of the order of $10e + 100$ instead of $10e - 50$). That is why we tried to find

another way of expressing the derivatives of the function in order to use them inside the Truncated Power Series Algebra. So, to further analyze $w(z)$ and related functions, we define two sequences of polynomials, $p_n(z)$ and $q_n(z)$, that follow specific recursive rules. The definitions are structured as follows:

1. The initial values for the sequences are:

$$p_0 = 1, \quad q_1 = 1, \quad p_n = 0 \text{ for } n < 0, \quad \text{and} \quad q_n = 0 \text{ for } n < 1.$$

2. For $n \geq 1$, the polynomials $p_n(z)$ are defined recursively as:

$$p_n = -2z \cdot p_{n-1} + p'_{n-1},$$

where p'_{n-1} denotes the derivative of p_{n-1} with respect to z .

3. The polynomials $q_n(z)$ are defined recursively as a linear combination of the $p_k(z)$ polynomials:

$$q_n = \sum_{k=0}^{n-1} c_k \cdot p_k,$$

where each coefficient c_k depends on the previous recursive steps and the properties of q_n and p_n .

Using these recursive definitions:

Definition 2.4.1. *The polynomial p_n is defined as follows:*

$$p_n = \begin{cases} 0 & \text{if } n < 0, \\ 1 & \text{if } n = 0, \\ -2z \cdot p_{n-1} + p'_{n-1} = -2z \cdot p_{n-1} - 2(n-1)p_{n-2} & \text{for } n \geq 1, \end{cases}$$

Definition 2.4.2. *The polynomial q_n is defined as follows:*

$$q_n = \begin{cases} 0 & \text{if } n < 1, \\ 1 & \text{if } n = 1, \\ \sum_{i=0}^{n-1} 2^{n-i-1} \frac{i!}{(2i-n+1)!} \cdot p_{2i-n+1} & \text{for } n \geq 2. \end{cases}$$

Lemma 2.4.3. *The polynomial q_n respects the relation*

$$q_{n+1} = -2zq_{n+1} - 2(n+1)q_n \tag{2.18}$$

Proof. Easy to prove by induction (but tedious). An alternative is to prove the next theorem deductively

and then use the theorem to prove that the polynomial q_n must satisfy this relation. $\square$

Theorem 2.4.4. *The solution of the nonlinear recursive relation*

$$w^{(n+2)} = -2zw^{(n+1)} - 2(n+1)w^{(n)} \quad n = 0, 1, \dots$$

is

$$w^{(n)}(z) = p_n(z)w(z) + q_n(z)\frac{2i}{\sqrt{\pi}} \quad (2.19)$$

Proof. Starting from the recursive equation we need to check that this is actually a solution. We do it by induction.

Base Case: For $n = 1$

$$w'(z) = -2zw(z)m + \frac{2i}{\sqrt{\pi}} = p_1(z)w(z) + q_1(z)\frac{2i}{\sqrt{\pi}}$$

(Strong) Induction Hypothesis: Assume 2.19 is a solution $\forall i < n + 1$, then we need to prove that it is a solution for $n + 2$

$$\begin{aligned} w^{(n+2)} &= -2zw^{(n+1)} - 2(n+1)w^{(n)} \\ &= -2z\left(p_{n+1}(z)w(z) + q_{n+1}(z)\frac{2i}{\sqrt{\pi}}\right) - 2(n+1)\left(p_n(z)w(z) + q_n(z)\frac{2i}{\sqrt{\pi}}\right) \\ &= w(z)\left(-2zp_{n+1}(z) - 2(n+1)p_n(z)\right) + \frac{2i}{\sqrt{\pi}}\left(-2zq_{n+1}(z) - 2(n+1)q_n(z)\right) \\ &= p_{n+2}(z)w(z) + q_{n+2}\frac{2i}{\sqrt{\pi}} \end{aligned}$$

where we have used the fact that $q_{n+2} = -2zq_{n+1} - 2(n+1)q_n$ and $p_{n+2} = -2zp_{n+1} - 2(n+1)p_n$. $\square$

Now 2.19 is numerically much more stable but when tested with very large values we will face an important difficulty. Indeed, if we take the values inside complex domain $A = \{z = re^{i\theta} \in \mathbb{C} \mid -\frac{\pi}{4} < \theta < \frac{5}{4}\pi\}$ the function and the derivatives are going to 0 very quickly as z becomes larger and larger. Unfortunately, in our expression 2.19 both terms $p_n(z)w(z)$ and $q_n\frac{2i}{\sqrt{\pi}}$ are huge when evaluated with a large $|z|$ with opposite sign. Basically we end up subtracting quantities of the order of $10e + 100$ and we expect to get a difference of the order of $10e - 40$. Thus, in the domain A we look for a perturbative approach. Especially we exploit a known expansion of the Faddeeva function (Abramowitz, M., and Stegun, I. A.) we have that at $z \rightarrow \infty$, the asymptotic expansion is:

$$w(z) \sim \frac{i}{\sqrt{\pi}z} \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2z^2)^n},$$

where $(2n-1)!!$ is the double factorial defined as:

$$(2n-1)!! = (2n-1)(2n-3)\cdots 3\cdot 1, \quad (0!! = 1).$$

Expanding the series explicitly:

$$w(z) \sim \frac{i}{\sqrt{\pi}z} \left(1 + \frac{1}{2z^2} + \frac{3}{(2z^2)^2} + \frac{15}{(2z^2)^3} + \cdots \right).$$

The dominant term as $|z| \rightarrow \infty$ is:

$$w(z) \sim \frac{i}{\sqrt{\pi}z}.$$

Thus we can derive the series to get all the derivatives we want (when doing this one should always check if the hypotheses to exchange the derivation and the infinite summation hold). If we proceed we get

$$\begin{aligned} w^{(0)}(z) &= \frac{i}{\pi\pi} \sum_{n=0}^{\infty} \frac{(2mn-1)!!}{2^n} z^{-2n} \\ w^{(1)}(z) &= -\frac{i}{\sqrt{\pi}} \sum_{mn=0}^{\infty} \frac{(2n-1)z^{-(2n+2)}(2n+1)}{2^n} \\ w^{(2)}(z) &= +\frac{i}{\sqrt{\pi}} \sum_{m=0}^{\infty} \frac{(2m-1)!2^{-(2m+3)}(2n+1)(2m+2)}{2^m} \\ &\vdots \\ w^{(m)}(z) &= (-1)^m \frac{i}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(2n-1)!!}{2^n} z^{-(2n+m)} \frac{(2n+m)!}{(2n)!} \end{aligned} \tag{2.20}$$

This approach is really stable and accurate (machine precision) for $|z| > 10$ in A and can be fine-tuned in order to consider only the relevant coefficient (in the code we take up to the seventh coefficient).

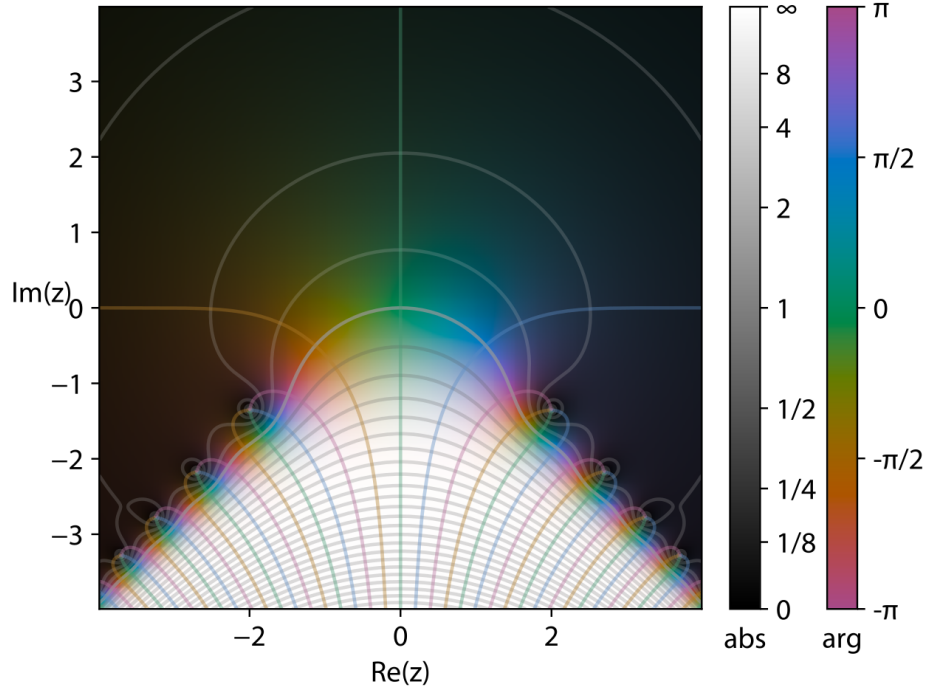


Figure 2.1: Values of the Faddeeva function inside the complex plane.

2.5 Sinc and Sinh

In this section we present how we dealt with the instabilities of $\text{sinc}(x) = \frac{\sin(x)}{x}$ and $\text{sinhc}(x) = \frac{\sinh(x)}{x}$. The instabilities are generated by the division by x when we get near to 0. This problem can be solved by a perturbative approach since it is related to only a single point where the functions are differentiable an infinite amount of times. In doing so our strategy is as follows: we expand the functions and their derivatives near to 0 and then we follow the same procedure of 2.20. As for the Faddeeva function expansion, we notice that to obtain continuity at $1e-16$ it is more than sufficient to store the first 7 coefficients of the expansions (it corresponds to going up to the 13th order) in the domain of this perturbative approach. In this case, for both functions, the domain is

$$A = \{z \in \mathbb{C} \mid |z| < 0.75\}$$

Thus, for sinc we have:

$$\begin{aligned}
\text{sinc}^{(0)}(x) &= \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n+1)!} \\
\text{sinc}^{(1)}(x) &= \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n-1}}{(2n+1)!} 2n \\
\text{sinc}^{(2)}(x) &= \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n-2}}{(2n+1)!} 2n(2n-1) \\
&\vdots \\
\text{sinc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} (-1)^n \frac{x^{2n-m}}{(2n+1)!} \frac{(2n)!}{(2n-m)!} = \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} (-1)^n \frac{x^{2n-m}}{(2n+1)(2n-m)!}
\end{aligned} \tag{2.21}$$

Something similar holds for sinhc:

$$\begin{aligned}
\text{sinhc}^{(0)}(x) &= \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n+1)!} \\
\text{sinhc}^{(1)}(x) &= \sum_{n=0}^{\infty} \frac{x^{2n-1}}{(2n+1)!} 2n \\
\text{sinhc}^{(2)}(x) &= \sum_{n=0}^{\infty} \frac{x^{2n-2}}{(2n+1)!} 2n(2n-1) \\
&\vdots \\
\text{sinhc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{x^{2n-m}}{(2n+1)!} \frac{(2n)!}{(2n-m)!} = \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{x^{2n-m}}{(2n+1)(2n-m)!}
\end{aligned} \tag{2.22}$$

This approach is really stable and computationally efficient because the terms of the summation are rapidly decreasing.

2.6 Asinc and Asinhc

In this section we explore a method to avoid the instabilities for $\text{asinc} = \frac{\arcsin(x)}{x}$ and $\text{arcsinhc} = \frac{\text{arcsinh}(x)}{x}$. Since the instabilities occur due to the division by x near to 0, we will proceed in a similar manner to sinc and sinhc (actually Asinhc has one more source of instability due to the singularity of the derivatives in 1). In this, as done before, we first compute the expansion around 0 and then we will

compute the derivatives

$$\begin{aligned}
\text{asinc}^{(0)}(x) &= \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n}}{2n+1} \\
\text{asinc}^{(1)}(x) &= \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n-1}}{2n+1} 2n \\
\text{asinc}^{(2)}(x) &= \sum_{n=0}^{\infty} \frac{(2n-1)!}{(2n)!!} \frac{x^{2n-2}}{2n+1} 2n(2n-1) \\
&\vdots \\
\text{asinc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n-m}}{2n+1} \frac{(2n)!}{(2n-m)!} = \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{((2n-1)!!)^2}{(2n-m)!} \frac{x^{2n-m}}{2n+1}
\end{aligned} \tag{2.23}$$

With the same approach we get as well the derivatives of asinhc

$$\begin{aligned}
\text{asinhc}^{(0)}(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n (2n-1)!!}{(2n)!!} \frac{x^{2n}}{2n+1} \\
\text{asinhc}^{(1)}(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n (2n-1)!!}{(2n)!!} \frac{x^{2n-1}}{2n+1} 2n \\
\text{asinhc}^{(2)}(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n (2n-1)!}{(2n)!!} \frac{x^{2n-2}}{2n+1} 2n(2n-1) \\
&\vdots \\
\text{asinhc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{(-1)^n (2n-1)!!}{(2n)!!} \frac{x^{2n-m}}{2n+1} \frac{(2n)!}{(2n-m)!} = \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{(-1)^n ((2n-1)!!)^2}{(2n-m)!} \frac{x^{2n-m}}{2n+1}
\end{aligned} \tag{2.24}$$

In this case, we can see that the terms of the summation, as n increases, are decreasing but with a slower rate with respect to the previous case of sinc and sinhc: thus we will check experimentally what is the threshold of convergence to get the best accuracy we can achieve. and then we will stick with those number of terms (one can also do it by hand estimating the error of the reminder of the summation as we did for the previous case of sinc and sinhc).

Chapter 3

Code and Results

In this chapter, we discuss the implementation of the newly derived series and the numerical results obtained using them. These results are compared with those generated by previously established methods. For this analysis, we utilize a software developed at CERN called *MAD-NG*.

MAD-NG (Methodical Accelerator Design—Next Generation) is an advanced computational tool used for designing and simulating particle accelerators. It is specifically tailored for high-performance computing, offering improved scalability and precision compared to its predecessors. The software incorporates cutting-edge algorithms for beam dynamics, and nonlinear optics, making it an essential tool for accelerator physicists.

More information about *MAD-NG* can be found at the following link: <https://mad.web.cern.ch/mad/>. Specifically, *MAD-NG* is providing us the framework where we are implementing the new series and comparing them to a 'naive approach' (actually sometimes is not so naive). As a first step we will discuss the data generation procedure to test the functions. After this first description we will move to the performance analysis of the function focusing on the regions where the naive approach presented problems of accuracy and instabilities.

3.1 Data Generation and Testing Framework

3.1.1 Data Generation

We first focus on the procedure we followed to generate the data. We need to rely on software that are designed to have a sufficient precision. Indeed it is important to remind that inside our framework, all the calculation should have at least $1e-16$ precision since we are not performing any approximation (except when dealing with perturbative approaches). That is why, the decision for the data generation

framework ends on symbolic calculation softwares. Specifically we used Wolfram Mathematica 14.1 and the python package 'sympy'. The reason for using two packages relies on the fact that we use these tools as a black box and we cannot check if their answer is correct or not if we do not have any other comparison means. For this reason it is important to double check the data generated, also because, on the cases analyzed in this work, both Mathematica and Sympy are facing various problems due to the unstable nature of the problems we are trying to solve (for example, both packages have serious problems when dealing with the Faddeeva function or sinc, sinhc etc...). In those cases we needed to rely on something (i.e. perturbative analyses) to understand what was the correct behaviour of the functions to later find a way of making both packages work: for example, for the Faddeeva function, using the standard definitions will lead to having severe numerical issues; that is why we had to rephrase the expression to be evaluated finding, in the end, the correct answer.

3.1.2 Sympy code

```
import sympy as sp
from sympy import I

def compute_and_evaluate_derivatives(x0_points, n, fun_expr, precision=25):
    """
    Computes Taylor series coefficients up to order n for a function expanded around each x0 in x0_points.

    Parameters:
    - x0_points: List of points at which to expand the Taylor series.
    - n: Order up to which to compute the Taylor series.
    - fun_expr: The function expression to expand (e.g., sp.asin(x)/x).
    - precision: Number of decimal places for the output.

    Returns:
    - List of lists of derivatives evaluated at each expansion point.
    """
    x = sp.symbols('x')
    all_derivatives = []

    for x0 in x0_points:
        derivatives_at_x0 = []
        for i in range(n + 1):
            # Compute the i-th derivative of the function
            derivative = sp.diff(fun_expr, x, i)

            # Divide by factorial(i)
            derivative /= sp.factorial(i)

            # Evaluate the derivative at x0
            derivative_at_x0 = derivative.subs(x, x0).evalf(precision + 5) # Extra precision
            derivatives_at_x0.append(derivative_at_x0)

        all_derivatives.append(derivatives_at_x0)

    return all_derivatives

def format_x0_as_pi_notation(x0):
    """
    Formats x0 in terms of multiples of pi if applicable, otherwise formats as a number.
    """
    if sp.simplify(x0 / sp.pi).is_rational:
        coeff = sp.simplify(x0 / sp.pi)
        return f"pi*{coeff}".replace('*1/', '/') if coeff != 1 else "pi"
    else:
        # Handle complex numbers
        real_part = sp.re(x0)
        imag_part = sp.im(x0)

        real_str = (f"pi*{sp.simplify(real_part / sp.pi)}" if sp.simplify(real_part / sp.pi).is_rational
```

```

        else str(real_part.evalf()).replace('*1/', '/')
    imag_str = (f"pi*{sp.simplify(imag_part / sp.pi)}" if sp.simplify(imag_part / sp.pi).is_rational
               else str(imag_part.evalf()).replace('*1/', '/'))

    if real_part == 0:
        return f"{imag_str}*1i"
    elif imag_part == 0:
        return real_str
    else:
        return f"{real_str} + {imag_str}*1i" if imag_part > 0 else f"{real_str} - {imag_str[1:]}*1i"

def write_coefficients_to_file(all_derivatives, x0_points, filename, function_name):
    """
    Writes Taylor series coefficients to a file for multiple expansion points.

    Parameters:
    - all_derivatives: List of derivatives evaluated at each x0.
    - x0_points: List of expansion points.
    - filename: Output file path.
    - function_name: Name of the function for the file header.
    """
    with open(filename, 'w') as file:
        # Write the header with the function name
        file.write(f"M.fun.{function_name} = {{\n")

        for i, derivatives in enumerate(all_derivatives):
            x0 = x0_points[i]

            # Format x0 in pi notation if possible
            x0_str = format_x0_as_pi_notation(x0)
            file.write(f"{{ x0 = {x0_str},\n")

            for j, derivative in enumerate(derivatives):
                if j % 2 == 0 and j != 0:
                    file.write("\n")

                # Check if the derivative is real or complex
                if derivative.is_real:
                    # Write the real derivative in scientific notation
                    derivative_str = f"{float(derivative.evalf()):.25e}".replace('e+', 'e+0').replace('e-', 'e-0')
                else:
                    # Format the complex derivative
                    real_part = sp.re(derivative).evalf()
                    imag_part = sp.im(derivative).evalf()

                    real_str = f"{float(real_part):.25e}".replace('e+', 'e+0').replace('e-', 'e-0')
                    imag_str = f"{float(imag_part):.25e}".replace('e+', 'e+0').replace('e-', 'e-0')

                    if real_part != 0:
                        derivative_str = f"{real_str} + {imag_str}i" if imag_part >= 0 else f"{real_str} - {imag_str[1:]}i"
                    else:
                        derivative_str = f"{imag_str}i" if imag_part >= 0 else f"-{imag_str[1:]}i"

                    file.write(f"{{ {derivative_str},")

            file.write("\n},\n\n")

        file.write("}\n")

# Example usage:
if __name__ == "__main__":
    # Set of x0 points
    x0_points = [
        10,
    ]

    # Order of derivatives
    n = 15

    # Number of decimal places
    precision = 25

    x = sp.symbols('x')

    # Define the Faddeeva function using its integral representation
    t = sp.symbols('t')
    integral_part = sp.integrate(sp.exp(t**2), (t, 0, x))
    fun_expr = sp.exp(-x**2) * (1 + (2 * sp.I / sp.sqrt(sp.pi)) * integral_part)

```



```

# Compute and evaluate the derivatives at the specified points
all_derivatives = compute_and_evaluate_derivatives(x0_points, n, fun_expr, precision)

# Output filename
output_filename = 'C:\\Users\\anton\\Downloads\\function_derivatives_at_points.txt'

# Specify the name of the function
function_name = "wf"

# Write all derivatives to the file
write_coefficients_to_file(all_derivatives, x0_points, output_filename, function_name)
print(f"All derivatives written to {output_filename}")

print("All files generated successfully.")

```

The provided Python code performs symbolic computation to generate the Taylor series coefficients of a given function at specified expansion points. It is composed of several functions:

1. compute_and_evaluate_derivatives

This function computes the coefficients of the Taylor series up to a specified order n for a given symbolic function.

- **Inputs:**

- $x0_points$: A list of points at which the function is expanded.
- n : The maximum order of derivatives to compute.
- fun_expr : The symbolic expression of the function.
- $precision$: The number of decimal places for the output (default is 25).

- **Process:**

- For each expansion point x_0 , compute up to the i -th coefficient of the Taylor expansion of the function.
- Evaluate the coefficients at x_0 and store it with extended precision.

- **Output:** A list of Taylor series coefficients for each expansion point.

2. format_x0_as_pi_notation

This function formats the expansion points x_0 in terms of multiples of π , if possible.

- **For Real Points:** If x_0 is a rational multiple of π , it is expressed in terms of π . Otherwise, it is formatted as a numerical value.

- **For Complex Points:** The real and imaginary parts are separately formatted, with the imaginary part appended as $+i$ or $-i$.

3. write_coefficients_to_file

This function writes the computed Taylor series coefficients into a file.

- **Process:**
 - Each expansion point is labeled using its formatted representation (e.g., in terms of π).
 - Coefficients are written in scientific notation, with special handling for complex values (real and imaginary parts are separated).
- **Output:** A structured file where each expansion point and its coefficients are clearly listed. The format to save the data is chosen to respect the test suite for we are using (in Lua).

Code Workflow

1. Define the symbolic function (`fun_expr`) using SymPy.
2. Call `compute_and_evaluate_derivatives` to compute Taylor coefficients for the function at specified points.
3. Format the expansion points using `format_x0_as_pi_notation`.
4. Save the results to a file using `write_coefficients_to_file`.

This is the main code used to generate the data with an accuracy of 25 digits as a default value. After generating the data, they are compared with the ones generated by Mathematica and then we test our numerical implementations.

3.1.3 Performance Analysis

To test the performance of our functions, we sample points in the domain $A = \{z \in \mathbb{C} \mid |z| < 10\}$ plus points with large absolute value up to $10e+08$: specifically we put emphasis on very small values up to $x_0 = 1e-8$, values in the very neighborhood of 1 (1 is a transition point for many functions where they change behavior), points in the very neighborhood of i , specific multiples of π (they generate alternating sign in some series). Then, we check if the function remains stable with large values. After this general check, we inspect points specific to the functions we are dealing with such as singularities,

borders where they change behavior and so on.

To measure the performance we use an adaptive metric that is defined as

$$||x - y|| = \frac{|x - y|}{|\max(x, 1)|}$$

That is basically an adaptive relative error used to deal with very large and small numbers at the same time. The sense of this relies in the fact that, if we are subtracting $x = 1e-24$ and $y = 1e-25$, a classical relative error will tell us that the error is equal to 90% but we know that, in the floating point arithmetic, we are still achieving machine precision so it would be meaningless to say that the error is huge. After defining the norm, we proceed as follows:

- We create a TPSA object 'P', where we set the point of evaluation x_0 and the differential unit $= 1$ (this means the in our dual algebra we have the point $x = x_0 + 1\epsilon$);
- We evaluate a function f with the previously defined TPSA object.
- We create another TPSA object 'Q' that we fill with all right coefficients computed with Sympy (it means that we are creating the dual number point $x = \sum_{i=0}^N \frac{f^{(i)}(x_0)}{i!} \epsilon^i$ where N is the maximum order of our differential unit and it is usually set to $N = 15$);
- we compute $d = ||P - Q||$ using the metric we defined previously applying it component wise;
- we take the maximum of of the computed distance $||d||_\infty$ and represent it as a function of the machine precision eps , i.e. $\frac{||d||_\infty}{\epsilon}$.

Now we can move to the description of the implementation of the previous mathematical formula of Chapter 2.

3.2 Numerical implementation and Performance

In this section we are going to analyze the numerical implementation of the formulas described in the second chapter and we are going to show the main improvements with respect to the previous implementations. The following sub-sections will be organized as follows:

- Coding implementation
- Explanation of the code
- Presentation of the results in peculiar points

3.2.1 Atan

We now present the code of Atan: in the following function we have both the previous (OLD_SERIES) and the new approach. We explain only the main logic focusing on the mathematical part given that the focus of this work is on the mathematics behind. Moreover, explaining all the technicalities of the code will require much time.

```
1 void
2 FUN(atan) (const T *a, T *c) // checked for real and complex
3 {
4     assert(a && c); DBGFUN(<->);
5     ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6     NUM a0 = a->coef[0], f0 = atan(a0);
7
8     ord_t to = c->mo;
9     if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<->); return; }
10
11 #if OLD_SERIES
12     if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
13         // atan(x) = -i/2 ln((i-x) / (i+x)) ; pole = +/- 1i
14 #ifdef MAD_CTPSA_IMPL
15         ctpsa_t *tn = GET_TMPX(c), *td = GET_TMPX(c);
16         mad_ctpsa_axpb(-1, a, I, tn);
17         mad_ctpsa_axpb(1, a, I, td);
18         mad_ctpsa_logxdy(tn, td, c);
19         mad_ctpsa_scl(c, -I/2, c);
20 #else
21         ctpsa_t *tn = GET_TMPX(c), *td = GET_TMPX(c);
22         mad_ctpsa_cplx(a, NULL, td);
23         mad_ctpsa_axpb(-1, td, I, tn);
24         mad_ctpsa_seti(td, 0, 1, I);
25         mad_ctpsa_logxdy(tn, td, tn);
26         mad_ctpsa_scl(tn, -I/2, tn);
27         mad_ctpsa_real(tn, c);
28 #endif
29         REL_TMPX(td), REL_TMPX(tn); DBGFUN(<->); return;
30     }
31 #endif // OLD_SERIES
32
33     NUM ord_coef[to+1], a2 = a0*a0, f1 = 1/(1+a2), f2 = f1*f1, f4 = f2*f2;
34     // fill orders <= MANUAL_EXPANSION_ORD
35     switch(MIN(to, MANUAL_EXPANSION_ORD)) {
36     case 6: ord_coef[6] = -a0*(1 + a2*(-10./3 + a2)) *f4*f2; /* FALLTHRU */
37     case 5: ord_coef[5] = (1./5 + a2*(-2 + a2)) *f4*f1; /* FALLTHRU */
38     case 4: ord_coef[4] = -a0*(-1 + a2) *f4; /* FALLTHRU */
39     case 3: ord_coef[3] = (-1./3 + a2) *f2*f1; /* FALLTHRU */
40     case 2: ord_coef[2] = -a0 *f2; /* FALLTHRU */
41     case 1: ord_coef[1] = f1; /* FALLTHRU */
42     case 0: ord_coef[0] = f0; break;
43     }
44     assert(!"unexpected missing coefficients");
45     // fill orders > MANUAL_EXPANSION_ORD
46     for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
47         ord_t n = (o-3)/2+1;
48         int s = 2*(o & 1)-1; // o: even = -1, odd = 1
49         NUM v = 0;
50         for (ord_t i = 0; i <= n; ++i, s = -s) {
51             num_t c = s * mad_num_powi(2,o-2*i-1) * mad_num_binom(o-i-1,i);
52             v += c * NUMF(div)(NUMF(powi)(a0,o-2*i-1), NUMF(powi)(1+a2,o-i));
53         }
54         ord_coef[o] = v/o;
55     }
56
57     fun_taylor(a,c,to,ord_coef);
58     DBGFUN(<->);
59 }
60 }
```

- **Initialization and Basic Setup:** The function first computes $f_0 = \text{atan}(a_0)$, where a_0 is the real part of the dual number given in input.
- **Manual Expansion of the Taylor Series:** If the Taylor series order to is less than or equal to a predefined limit `MANUAL_EXPANSION_ORD`, which is set to 6, the function computes the Taylor coefficients manually. These coefficients are stored in the array `ord_coef`.
- **Old Approach (OLD_SERIES):** If the series order exceeds `MANUAL_EXPANSION_ORD`, the computation follows the identity:

$$\text{atan}(x) = -\frac{i}{2} \ln \left(\frac{i-x}{i+x} \right)$$

- **New Approach:** For Taylor series orders greater than `MANUAL_EXPANSION_ORD`, the formula presented in Chapter 2 (see Equation 2.13) is applied. The coefficients are calculated and stored in the array `ord_coef`. This is specifically done in the last 2 'for' loops.
- **Conclusion:** After calculating the Taylor series coefficients stored in `ord_coef`, the function evaluates the series as a truncated power series using the function `fun_taylor`. The output is then stored in `c`.

From a numerical point of view we are now showing the difference of performance in some specific and representative points between the previous and new approach. We present the errors as number of eps per order (if a specific order does not appear it means that the error is 0): on the left we have the errors obtained with the previous approach whereas on the right we have the ones obtained with the new approach.

Comparison at $x_0 = 0.1$

Point: $x_0 = 0.1$

Previous approach		
I	Coefficient Error(eps)	Order
1	$6.2500000000000000 \times 10^{-2}$	0
2	$3.1250000000000000 \times 10^{-1}$	2
3	$5.0000000000000000 \times 10^{-1}$	3
4	$2.4375000000000000 \times 10^{+0}$	4
5	$1.8750000000000000 \times 10^{+0}$	5
6	$5.9375000000000000 \times 10^{+0}$	6
7	$3.1812500000000000 \times 10^{+1}$	7
8	$2.7562500000000000 \times 10^{+1}$	8
9	$1.7162500000000000 \times 10^{+2}$	9
10	$2.2687500000000000 \times 10^{+2}$	10
11	$1.7715937500000000 \times 10^{+3}$	11
12	$1.3632500000000000 \times 10^{+3}$	12
13	$8.4610156250000000 \times 10^{+3}$	13
14	$3.0157875000000000 \times 10^{+4}$	14
15	$8.4493683593750000 \times 10^{+4}$	15

Maximum Error: 84494eps, at
 $x_0 = 0.1$ observed at order 15
(missing orders are equal to 0).
From the two

New approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^{+0}$	0
2	$6.2500000000000000 \times 10^{-2}$	2
3	$2.5000000000000000 \times 10^{-1}$	3
4	$6.2500000000000000 \times 10^{-2}$	4
5	$1.2500000000000000 \times 10^{-1}$	5
6	$0.0000000000000000 \times 10^{+0}$	6
7	$1.2500000000000000 \times 10^{-1}$	7
8	$0.0000000000000000 \times 10^{+0}$	8
9	$6.2500000000000000 \times 10^{-2}$	9
10	$6.2500000000000000 \times 10^{-2}$	10
11	$1.2500000000000000 \times 10^{-1}$	11
12	$6.2500000000000000 \times 10^{-2}$	12
13	$1.5625000000000000 \times 10^{-2}$	13
14	$0.0000000000000000 \times 10^{+0}$	14
15	$1.5625000000000000 \times 10^{-2}$	15

Maximum Error: 1eps, at
 $x_0 = 0.1$ observed at order 3
(missing orders are equal to 0).

The results shows that the new approach has an impressive accuracy, especially if compared to the previous approach. Moreover, going towards zero, the instability of the old approach further increases.

Comparison at $x_0 = \frac{\pi}{3}$

Point: $x_0 = \frac{\pi}{3}$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$1.4148475504056880 \times 10^{16}$	0
2	$1.1594725347858086 \times 10^1$	1
3	$0.0000000000000000 \times 10^0$	2
4	$3.2298691768100241 \times 10^1$	3
5	$0.0000000000000000 \times 10^0$	4
6	$5.9553733096040816 \times 10^1$	5
7	$0.0000000000000000 \times 10^0$	6
8	$9.5092423640831768 \times 10^1$	7
9	$0.0000000000000000 \times 10^0$	8
10	$1.5813216984796364 \times 10^1$	9
11	$0.0000000000000000 \times 10^0$	10
12	$3.5620189341245663 \times 10^2$	11
13	$0.0000000000000000 \times 10^0$	12
14	$4.9228172283738519 \times 10^2$	13
15	$0.0000000000000000 \times 10^0$	14
16	$1.8999872411265883 \times 10^3$	15

Maximum Error:
 $1.4148475504057 \times 10^{16}$ eps,
observed at order 0 (missing
orders are equal to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.1594725347858086 \times 10^1$	1
3	$0.0000000000000000 \times 10^0$	2
4	$3.3590639438824240 \times 10^1$	3
5	$0.0000000000000000 \times 10^0$	4
6	$5.5869997028244526 \times 10^1$	5
7	$0.0000000000000000 \times 10^0$	6
8	$7.7446406882739581 \times 10^1$	7
9	$0.0000000000000000 \times 10^0$	8
10	$9.9910780040302441 \times 10^1$	9
11	$0.0000000000000000 \times 10^0$	10
12	$1.2173988762196593 \times 10^2$	11
13	$0.0000000000000000 \times 10^0$	12
14	$1.4383540049982327 \times 10^2$	13
15	$0.0000000000000000 \times 10^0$	14
16	$1.6440488288519845 \times 10^2$	15

Maximum Error: 165 eps,
observed at order 15 (missing
orders are equal to 0).

Here we notice that the scalar part of the previous approach is totally off and moreover the coefficients are getting less and less accurate. For the new approach we notice that the stability is improved: even in this case we have inaccuracies for high orders but this is also due to the fact that we are near a singularity.

Comparison at $x_0 = 0.1i$

Point: $x_0 = 0.1i$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$2.5000000000000000 \times 10^{-1}$	3
3	$0.0000000000000000 \times 10^0$	4
4	$3.3750000000000000 \times 10^0$	5
5	$0.0000000000000000 \times 10^0$	6
6	$5.7250000000000000 \times 10^1$	7
7	$0.0000000000000000 \times 10^0$	8
8	$1.4725000000000000 \times 10^2$	9
9	$0.0000000000000000 \times 10^0$	10
10	$2.7786250000000000 \times 10^3$	11
11	$0.0000000000000000 \times 10^0$	12
12	$3.8330625000000000 \times 10^4$	13
13	$0.0000000000000000 \times 10^0$	14
14	$1.0859900000000000 \times 10^5$	15

Maximum Error: 108599eps,
observed at order 15 (missing
orders are equal to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.2500000000000000 \times 10^{-1}$	7
3	$0.0000000000000000 \times 10^0$	8
4	$1.2500000000000000 \times 10^{-1}$	9
5	$0.0000000000000000 \times 10^0$	10
6	$1.2500000000000000 \times 10^{-1}$	11
7	$0.0000000000000000 \times 10^0$	12
8	$1.2500000000000000 \times 10^{-1}$	13
9	$0.0000000000000000 \times 10^0$	14
10	$2.5000000000000000 \times 10^{-1}$	15

Maximum Error: 1eps, observed
at order 14 (missing orders are
equal to 0).

In this last case we see again that the new approach is outperforming the old one.

Overall, the new approach is much more stable and efficient, especially in the domain $\{z \in \mathbb{C} \mid \|z\| < 1.1\}$.

3.2.2 Acot, Atanh and Acoth

He we will not present the code used for the implementation of acot atanh and acoth since it is really similar to the one used for atan. Also, for the performance the improvements are really similar to the one obtained with the new implementation of atan: indeed all the new functions have a a much better accuracy in the domain $\{z \in \mathbb{C} \mid |z| < 1.3\}$ and also around singularities (around the singularities the new formulas are reducing the error but still presents inaccuracies; with 'near' we mean of distance $= 1e-06$). Thus, not to bore the reader, we put this brief analyses in the Appendix. It is still worth saying that the acot implementation is relying on the following equivalence

$$\text{acot} = \frac{i}{2} \log \left(\frac{x-i}{x+i} \right)$$

whereas for atanh and acoth , the formulas used are the standard one in the real domain.

3.2.3 Acosh

In this section we are going to test the performance of acosh . We start by describing the code.

```

1 void
2 FUN(acosh) (const T *a, T *c) // checked for real and complex
3 {
4     assert(a && c); DBGFUN(->);
5     ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6     NUM a0 = a->coef[0];
7     ensure(SELECT(a0 > 1, 1), "invalid domain acosh(FMT)", VAL(a0));
8     NUM f0 = acosh(a0);
9
10    ord_t to = c->mo;
11    if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }
12
13    #if OLD_SERIES
14    if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
15        // acosh(x) = ln(x + sqrt(x^2-1))
16        FUN(logaxpsqrtpcx2)(a, 1, -1, 1, c);
17        DBGFUN(<-); return;
18    }
19    #endif // OLD_SERIES
20
21    NUM ord_coef[to+1], a2 = a0*a0, f1 = 1/sqrt(a2-1), f2 = f1*f1, f4 = f2*f2;
22    // fill orders <= MANUAL_EXPANSION_ORD
23    switch(MIN(to, MANUAL_EXPANSION_ORD)) {
24    case 6: ord_coef[6] = -a0*(5./16 + a2*(5./6 + 1./6*a2)) *f4*f4*f2*f1; /* FALLTHRU */
25    case 5: ord_coef[5] = (3./40 + a2*(3./5 + 1./5*a2)) *f4*f4*f1; /* FALLTHRU */
26    case 4: ord_coef[4] = -a0*(3./8 + 1./4*a2) *f4*f2*f1; /* FALLTHRU */
27    case 3: ord_coef[3] = (1./6 + 1./3*a2) *f4*f1; /* FALLTHRU */
28    case 2: ord_coef[2] = -a0*(1./2) *f2*f1; /* FALLTHRU */
29    case 1: ord_coef[1] = f1; /* FALLTHRU */
30    case 0: ord_coef[0] = f0; break;
31    }
32    assert(!"unexpected missing coefficients");
33    // fill orders > MANUAL_EXPANSION_ORD
34    if (to > MANUAL_EXPANSION_ORD) {
35        NUM _a0p1 = 1/(a0+1);
36        NUM _a0m1 = 1/(a0-1);
37        NUM den = NUMF(powi)(-2,MANUAL_EXPANSION_ORD-1) * sqrt((a0+1)*(a0-1));
38
39        for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
40            int n = (o-1)/2+1;
41            NUM num = 0; den *= -2;
42            FOR(i,n) {
43                int d = o-2*i-1 ? 1 : 2;
44                num_t c = (i ? mad_num_fact2(2*i-1) : 1.) / d;
45                num += c * mad_num_fact2(2*o-2*i-3) * mad_num_binom(o-1,i) *
46                    (NUMF(powi)(_a0p1,o-i-1) * NUMF(powi)(_a0m1,i) +
47                     NUMF(powi)(_a0m1,o-i-1) * NUMF(powi)(_a0p1,i) );
48            }
49            ord_coef[o] = num/den/mad_num_fact(o);
50        }
51    }
52
53    fun_taylor(a,c,to,ord_coef);
54    DBGFUN(<-);
55 }
56
57

```

- **Initialization:**

- Extract a_0 (the scalar part of a) and compute $f_0 = \operatorname{acosh}(a_0)$.

- Ensure the domain condition $a_0 > 1$. If to, the truncation order, = 0 or a is a scalar, set c to f_0 and exit.
- **Old Approach (OLD_SERIES):** For $to > \text{MANUAL_EXPANSION_ORD}$, where to is the truncation order, the function uses the simplified formula:

$$\text{acosh}(x) = \ln(x + \sqrt{x^2 - 1}),$$

- **New Approach:**

- **Taylor Expansion (Orders $\leq \text{MANUAL_EXPANSION_ORD}$):** Precompute:

$$f_1 = \frac{1}{\sqrt{a_0^2 - 1}}, \quad f_2 = f_1^2, \quad f_4 = f_2^2,$$

and manually calculate coefficients for orders $k = 0, \dots, \text{MANUAL_EXPANSION_ORD}$ using closed-form expressions.

- **High-Order Terms (Orders $> \text{MANUAL_EXPANSION_ORD}$):** Use a recurrence relation to calculate:

$$\text{ord_coef}[o] = \frac{\text{num}}{\text{den} \cdot o!},$$

where num and den correspond to the numerator and denominator in the formula for $\text{acosh}(x)$ in Equation 2.17, computed iteratively.

- **Final Evaluation:** After computing all Taylor series coefficients, evaluate the series using `fun_taylor` and store the result in c .

In this case the new approach is aiming to solve the instabilities due to the singularity at $x = 1$. Overall we can see that the new method is improving the previous approach but the instability around the singularity is still there.

Comparison at $x_0 = 1.000001$

Point: $x_0 = 1.000001$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$1.2069042968750000 \times 10^2$	0
2	$8.5155137320644790 \times 10^4$	1
3	$2.5546406642617457 \times 10^5$	2
4	$4.2577345718131086 \times 10^5$	3
5	$5.9608299379351037 \times 10^5$	4
6	$7.6639147943352477 \times 10^5$	5
7	$9.3670105141980434 \times 10^5$	6
8	$1.1070112260683978 \times 10^6$	7
9	$1.2773205637951044 \times 10^6$	8
10	$1.4476289338701826 \times 10^6$	9
11	$1.6179393993359790 \times 10^6$	10
12	$1.7882479277186738 \times 10^6$	11
13	$1.9585590862990867 \times 10^6$	12
14	$2.1288677870886698 \times 10^6$	13
15	$2.2991750868642027 \times 10^6$	14
16	$2.4694843182200179 \times 10^6$	15

Maximum Error:

$2.4694843182200179 \times 10^6$ eps,
observed at order 15 (missing
orders are equal to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$2.3764583488195634 \times 10^0$	0
2	$7.7244657686125083 \times 10^2$	1
3	$2.3173403424030284 \times 10^3$	2
4	$8.4020411905578985 \times 10^3$	3
5	$1.1762854322072972 \times 10^4$	4
6	$1.5123684769709163 \times 10^4$	5
7	$1.8484500634255277 \times 10^4$	6
8	$2.1845308833148971 \times 10^4$	7
9	$2.5206135995561948 \times 10^4$	8
10	$2.8566951571758367 \times 10^4$	9
11	$3.1927770091780083 \times 10^4$	10
12	$3.5288575448213080 \times 10^4$	11
13	$3.8649404546716665 \times 10^4$	12
14	$4.2010227757040942 \times 10^4$	13
15	$4.5371035168825969 \times 10^4$	14
16	$4.8731851282699979 \times 10^4$	15

Maximum Error:

$4.8731851282699979 \times 10^4$ eps,
observed at order 15 (missing
orders are equal to 0).

As we can see, the new approach is improving the accuracy (by a factor 100) but still presents problems due to the singularity if their derivative. Other than in the neighborhood of 1, both approaches are performing very well, always achieving machine precision.

3.2.4 Faddeeva function

```

1 void
2 FUN(wf) (const T *a, T *c)
3 {
4     assert(a && c); DBGFUN(->);
5     ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6     NUM a0 = a->coef[0];
7     NUM f0 = NUMF(wf)(a0);
8
9     ord_t to = c->mo;
10    if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }
11
12    NUM ord_coef[to+1];
13    ord_coef[0] = f0;
14    ord_coef[1] = -2*a0*f0 + I*M_2_SQRTPI;
15

```

```

16  if (fabs(a0) <= 7 || (cimag(a0) < creal(a0)+0.5 && cimag(a0) < -creal(a0)+0.5)) {
17      NUM p[to+1], q[to+1];
18      p[0] = 1;
19      p[1] = -2*a0;
20      q[0] = 0;
21      q[1] = 1;
22
23      for (ord_t o = 2; o <= to; ++o) {
24          q[o] = 0;
25  #ifdef MAD_CTPSA_IMPL
26          q[o] = -2*(o-1)*q[o-2] - 2*a0*q[o-1];
27  #endif
28          p[o] = -2*(o-1)*p[o-2] - 2*a0*p[o-1];
29          ord_coef[o] = (p[o]*f0 + q[o]*I*M_2_SQRTPI)/mad_num_fact(o);
30      }
31
32  } else if (fabs(a0) > 7 && fabs(a0) < 100) { // adjusted for double precision
33  #ifdef MAD_CTPSA_IMPL
34      NUM a02 = a0*a0, a04=a02*a02, a06=a04*a02, a08=a04*a04;
35      NUM coef[7] = {
36          -I*M_SQRTPI/(a04),
37          -I*M_SQRTPI/(a06) *5,
38          -I*M_SQRTPI/(a06*a02)/4 *105,
39          -I*M_SQRTPI/(a06*a04)/4 *630,
40          -I*M_SQRTPI/(a08*a04)/16*17325,
41          -I*M_SQRTPI/(a08*a06)/16*135135,
42          -I*M_SQRTPI/(a08*a08)/64*4729725 };
43  #endif
44
45      ord_coef[2] = -a0*(-2*a0*f0 + I*M_2_SQRTPI) - f0;
46
47      for (ord_t o = 3; o <= to; ++o) {
48          FOR(i,7) ord_coef[o] += coef[i];
49          FOR(i,7) coef[i] *= -1/a0*(2.*i+o+1)/(o+1);
50      }
51  } else {
52      for (ord_t o = 1; o <= to; ++o)
53          ord_coef[o] = -ord_coef[o-1]/a0;
54  }
55
56  fun_taylor(a,c,to,ord_coef);
57  DBGFUN(<-);
58  }
59

```

- **Initialization and Basic Setup:** The function begins by extracting the zeroth-order coefficient $a_0 = a \rightarrow \text{coef}[0]$ and computing $f_0 = \text{wf}(a_0)$ (wf here is the name of the Faddeeva function). If the truncation order $\text{to} = 0$ or the input a is a scalar, the output series c is set to f_0 , and the function exits.
- **First Branch (Exact formula 2.19)** When $|a_0| \leq 7$ or $\text{Im}(a_0) < -|\text{Re}(a_0)| + 0.3$, a direct Taylor expansion is computed using 2.19:

$$p_0 = 1, \quad p_1 = -2a_0, \quad p_n = -2(n-1)p_{n-2} - 2a_0p_{n-1},$$

$$q_0 = 0, \quad q_1 = 1, \quad q_n = -2(n-1)q_{n-2} - 2a_0q_{n-1}.$$

The coefficients are derived as:

$$\text{ord_coef}[n] = \frac{p_n f_0 + q_n \frac{2i}{\sqrt{\pi}}}{n!}.$$

- **Second Branch (Perturbative Approach):** When $7 < |a_0| < 100$, we use the perturbative

approach (plus an exponential decaying correction; we did not present it before because it is not significant and we reach soon the underflow regime). The coefficients are computed using a combination of precomputed terms and numerical adjustments. Specifically:

the Taylor coefficients are evaluated as:

$$\text{ord_coef}[n] = \sum_k \text{coef}[k],$$

where the precomputed terms $\text{coef}[k]$ are iteratively updated.

- **Third Branch (Simplified Perturbative Approach):** For $|a_0| \geq 100$, the computation is simplified. Coefficients are determined via:

$$\text{ord_coef}[n] = -\frac{\text{ord_coef}[n-1]}{a_0}.$$

This reduces complexity while maintaining stability for large $|a_0|$. In this scenario we keep only the first order term of the perturbative approach; this is possible thanks to the fast decaying of the higher order terms.

- **Finalization:** The computed coefficients ord_coef are used to evaluate the Taylor series via fun_taylor , and the result is stored in c , completing the computation.

It is worth saying that the previous approach was based on the identity

$$w(z) = \text{erfcx}(-iz), \quad \text{erfcx} = e^{-z^2} (1 - \text{erf}(-iz))$$

Comparison at $x_0 = 1$

Point: $x_0 = 1$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$4.5037732096678219 \times 10^{14}$	1
3	$4.4532746461811550 \times 10^{15}$	2
4	$5.6139482103745156 \times 10^{14}$	3
5	$4.4622786780737570 \times 10^{15}$	4
6	$5.8348081431391738 \times 10^{14}$	5
7	$4.4638480761692070 \times 10^{15}$	6
8	$5.8917354889630300 \times 10^{14}$	7
9	$4.4642737891667770 \times 10^{15}$	8
10	$5.9096169669658250 \times 10^{14}$	9
11	$4.4644155285850945 \times 10^{15}$	10
12	$5.9160650432015550 \times 10^{14}$	11
13	$4.4644691621280820 \times 10^{15}$	12
14	$5.9186362426286275 \times 10^{14}$	13
15	$4.4644913884787495 \times 10^{15}$	14
16	$5.9197433000164788 \times 10^{14}$	15

Maximum Error:
 $4.5037732096678 \times 10^{14}$ eps,
observed at order 1 (missing
orders are equal to 0)

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$5.8774717541114375 \times 10^{-39}$	7
3	$1.1754943508222875 \times 10^{-38}$	8
4	$0.0000000000000000 \times 10^0$	9
5	$4.7019774032891500 \times 10^{-38}$	10
6	$0.0000000000000000 \times 10^0$	11
7	$1.8807909613156600 \times 10^{-37}$	12
8	$0.0000000000000000 \times 10^0$	13
9	$0.0000000000000000 \times 10^0$	14
10	$3.7615819226313200 \times 10^{-37}$	15

Maximum Error: 0eps, observed
at order 3 (missing orders are
equal to 0).

Here we are comparing the first branch of the new method with the previous approach.

Comparison at $x_0 = 0.71$

Point: $x_0 = 0.71$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$6.8815885555358160 \times 10^{15}$	1
3	$-5.1591594667756550 \times 10^{14}$	2
4	$-4.5517434546355230 \times 10^{15}$	3
5	$3.8357952196568681 \times 10^{14}$	4
6	$4.4765561162969125 \times 10^{15}$	5
7	$-4.1469840325789381 \times 10^{14}$	6
8	$-4.4803764120651815 \times 10^{15}$	7
9	$4.3117511363482631 \times 10^{14}$	8
10	$4.4813883397505280 \times 10^{15}$	9
11	$-4.3688034614841488 \times 10^{14}$	10
12	$-4.4817336255584970 \times 10^{15}$	11
13	$4.3904796101476856 \times 10^{14}$	12
14	$4.4818701826899785 \times 10^{15}$	13
15	$-4.3994745862836444 \times 10^{14}$	14
16	$-4.4819288125781150 \times 10^{15}$	15

Maximum Error:
 $6.8815885555358 \times 10^{15}$ eps,
observed at order 1.

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.2500000000000000 \times 10^{-1}$	3
3	$2.5000000000000000 \times 10^{-1}$	4
4	$1.5625000000000000 \times 10^{-2}$	5
5	$0.0000000000000000 \times 10^0$	6
6	$0.0000000000000000 \times 10^0$	7
7	$0.0000000000000000 \times 10^0$	8
8	$0.0000000000000000 \times 10^0$	9
9	$0.0000000000000000 \times 10^0$	10
10	$1.9531250000000000 \times 10^{-3}$	11
11	$1.2207031250000000 \times 10^{-4}$	12
12	$0.0000000000000000 \times 10^0$	13
13	$0.0000000000000000 \times 10^0$	14
14	$1.5258789062500000 \times 10^{-5}$	15

Maximum Error: 1 eps, observed
at order 4.

Here we compare the second branch of the function near to the border of the domain of validity of this new branch.

Comparison at $x_0 = 100$

Point: $x_0 = 100$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$4.5035996442635000 \times 10^{13}$	1
3	$4.5030930072371470 \times 10^{15}$	2
4	$5.6293412323324141 \times 10^{13}$	3
5	$4.5031848984217345 \times 10^{15}$	4
6	$5.8544775615491875 \times 10^{13}$	5
7	$4.5032008586995415 \times 10^{15}$	6
8	$5.9125765535819336 \times 10^{13}$	7
9	$4.5032051851921425 \times 10^{15}$	8
10	$5.9308320363891961 \times 10^{13}$	9
11	$4.5032066254084640 \times 10^{15}$	10
12	$5.9374156837077625 \times 10^{13}$	11
13	$4.5032071703442730 \times 10^{15}$	12
14	$5.9400410470472359 \times 10^{13}$	13
15	$4.5032073961661765 \times 10^{15}$	14
16	$5.9411714436177109 \times 10^{13}$	15

Maximum Error:
45035996442635eps, observed at
order 1 (missing orders are equal
to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$0.0000000000000000 \times 10^0$	1

Maximum Error: 0eps, observed
at order 1 (missing orders are
equal to 0).

Here we are using the third branch of the new method.

Comparison at $x_0 = 1e - 06$

Point: $x_0 = 1e - 06$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.9170735247653172 \times 10^{10}$	1
3	$5.7067278326545550 \times 10^{15}$	2
4	$1.0083194689236149 \times 10^{10}$	3
5	$4.5543096909906270 \times 10^{15}$	4
6	$9.0037640752461815 \times 10^9$	5
7	$4.5045411221068900 \times 10^{15}$	6
8	$8.9115142025309219 \times 10^9$	7
9	$4.5036097222366470 \times 10^{15}$	8
10	$8.8904891547323952 \times 10^9$	9
11	$4.5035996984701370 \times 10^{15}$	10
12	$8.8831016123221226 \times 10^9$	11
13	$4.5035996277174280 \times 10^{15}$	12
14	$8.8801651707066174 \times 10^9$	13
15	$4.5035996273630805 \times 10^{15}$	14
16	$8.8789022410361176 \times 10^9$	15

Maximum Error:
19170735248eps, observed at
order 1 (missing orders are equal
to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$5.0000000000000000 \times 10^{-1}$	2
3	$0.0000000000000000 \times 10^0$	3
4	$2.5000000000000000 \times 10^{-1}$	4
5	$0.0000000000000000 \times 10^0$	5
6	$1.2500000000000000 \times 10^{-1}$	6
7	$0.0000000000000000 \times 10^0$	7
8	$3.1250000000000000 \times 10^{-2}$	8
9	$0.0000000000000000 \times 10^0$	9
10	$7.8125000000000000 \times 10^{-3}$	10
11	$1.4901161193847656 \times 10^{-8}$	11
12	$1.9531250000000000 \times 10^{-3}$	12
13	$0.0000000000000000 \times 10^0$	13
14	$3.6621093750000000 \times 10^{-4}$	14
15	$2.3283064365386963 \times 10^{-10}$	15

Maximum Error: 1 eps, observed
at order 1 (missing orders are
equal to 0).

Here we check the stability of the first branch when we are very near to 0.

Comparison at $x_0 = -10i$

Point: $x_0 = -10i$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.5537418714428211 \times 10^{17}$	2
3	$0.0000000000000000 \times 10^0$	3
4	$4.1143740543120064 \times 10^{17}$	4
5	$0.0000000000000000 \times 10^0$	5
6	$3.1577171737004077 \times 10^{17}$	6
7	$0.0000000000000000 \times 10^0$	7
8	$1.0558108753203880 \times 10^{17}$	8
9	$0.0000000000000000 \times 10^0$	9
10	$2.1626346290848756 \times 10^{16}$	10
11	$0.0000000000000000 \times 10^0$	11
12	$6.2201833421438140 \times 10^{15}$	12
13	$0.0000000000000000 \times 10^0$	13
14	$4.6145211288905130 \times 10^{15}$	14

Maximum Error:
 $4.114374054312 \times 10^{17}$ eps,
observed at order 4 (missing
orders are equal to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$-5.2830545805637907 \times 10^{-1}$	2
3	$0.0000000000000000 \times 10^0$	3
4	$0.0000000000000000 \times 10^0$	4
5	$-0.0000000000000000 \times 10^0$	5
6	$0.0000000000000000 \times 10^0$	6
7	$0.0000000000000000 \times 10^0$	7
8	$5.9813804573103313 \times 10^{-1}$	8
9	$-0.0000000000000000 \times 10^0$	9
10	$0.0000000000000000 \times 10^0$	10
11	$0.0000000000000000 \times 10^0$	11
12	$5.9452414065727122 \times 10^{-1}$	12

Maximum Error: 1 eps, observed
at order 8 (missing orders are
equal to 0).

Here we check a negative imaginary number because the function is switching behavior in this part of the complex plane (see 2.1).

Comparison at $x_0 = 1e+07$

Point: $x_0 = 1e+07$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$4.5035996273704958 \times 10^9$	1
3	$4.5035996273654295 \times 10^{15}$	2
4	$5.6294995342115374 \times 10^9$	3
5	$4.5035996273663485 \times 10^{15}$	4
6	$5.8546795155796251 \times 10^9$	5
7	$4.5035996273665085 \times 10^{15}$	6
8	$5.9127904785133171 \times 10^9$	7
9	$4.5035996273665515 \times 10^{15}$	8
10	$5.9310497835747757 \times 10^9$	9
11	$4.5035996273665660 \times 10^{15}$	10
12	$5.9376348165198689 \times 10^9$	11
13	$4.5035996273665715 \times 10^{15}$	12
14	$5.9402607334679136 \times 10^9$	13
15	$4.5035996273665735 \times 10^{15}$	14
16	$5.9413913685908995 \times 10^9$	15

Maximum Error:
4503599628eps, observed at
order 1 (missing orders are equal
to 0).

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0

Maximum Error: 0eps, observed
at order 1 (missing orders are
equal to 0).

Here we are checking the results for a very large value (we are in the third branch).

The function has been tested in many more points, focusing especially on the borders of the various domains that are creating the three branches (an improvement to be done is to refine the method around the $1D$ domain defined by $\text{Im}(z) = -|\text{Re}(z)|$ where we lose some accuracy). Overall we can say that the new method is really precise and stable and it is largely outperforming the previous one.

3.2.5 Sinc

Now we are analyzing the sinc function: here the problem was only related to $x_0 = 0$ and the instability is caused by the division.

```

1 void
2 FUN(sinc) (const T *a, T *c)

```

```

3 {
4   assert(a && c); DBGFUN(->);
5   ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6
7   NUM a0 = a->coef[0];
8   NUM f0 = NUMF(sinc)(a0);
9   ord_t to = c->mo;
10
11   if (!to || FUN(isval)(a)) {
12     FUN(setval)(c,f0); DBGFUN(<-); return;
13   }
14
15   if (fabs(a0) > 0.75) { // sinc(x), |x| > 0.75
16     T *t = GET_TMPX(c);
17     FUN(sin)(a,t);
18     FUN(div)(t,a,c);
19     FUN(seti)(c,0,0,f0); // scalar part not very accurate
20     REL_TMPX(t); DBGFUN(<-); return;
21   }
22
23   NUM ord_coef[to+1];
24
25   if (fabs(a0) > 1e-12) { // sinc(x), 1e-12 < |x| <= 0.75
26     num_t odd_coef[7] = {
27       -3, 30, -840, 45360, -3991680, 518918400, -93405312000
28     };
29     int s = 1; // o/2: even = 1, odd = -1
30     num_t fact = 1;
31     for (ord_t o = 1; o <= to; o += 2, s = -s) {
32       NUM v1 = 0, v2 = 0;
33       FOR(i,7) {
34         v1 += 1 / odd_coef[i] * NUMF(powi)(a0,2*i+1);
35         v2 += 1 / odd_coef[i] * NUMF(powi)(a0,2*i)*(2*i+1);
36       }
37       fact *= o*(o+1.);
38       ord_coef[o] = s * v1 / fact*(o+1);
39       ord_coef[(o+1)%(to+1)] = s * v2 / fact;
40       if (o+2 <= to) {
41         static const num_t stp_coef[7] = {
42           -2, 12, -240, 10080, -725760, 79833600, -12454041600
43         };
44         FOR(i,7) odd_coef[i] += stp_coef[i];
45       }
46     }
47     ord_coef[0] = f0;
48   } else { // sinc(x), |x| <= 1e-12
49     ord_coef[0] = 1;
50     ord_coef[1] = 0;
51     for (ord_t o = 2; o <= to; ++o)
52       ord_coef[o] = -ord_coef[o-2] / (o*(o+1.));
53   }
54
55   fun_taylor(a,c,to,ord_coef);
56   DBGFUN(<-);
57 }

```

Key Variables and Arrays

- a_0 : Zeroth coefficient of a , representing the point at which the Taylor series is expanded.
- `ord_coef[i]`: Array storing the Taylor series coefficients for each order $i \in \{0, \dots, \text{to}\}$.
- `odd_coef0`: Auxiliary array of coefficients.

`odd_coef0` = $[-3, 30, -840, 45360, -3991680, 518918400, -93405312000]$.

- **add_odd:** Incremental coefficients for updating odd_coef:

$$\text{add_odd} = [2, 12, -240, -10080, -725760, -79833600, -12454041600].$$

Case Analysis

1. Large $|a_0|$ ($|a_0| > 0.75$)

In this regime, the function directly computes $\text{sinc}(x)$ using its definition:

$$\text{sinc}(x) = \frac{\sin(x)}{x}.$$

- Compute $\sin(a)$ and store the result in a temporary variable t .
- Divide t by a to obtain $\text{sinc}(a)$, and store this in c .
- Adjust the scalar part ($c[0]$) to match $f_0 = \text{sinc}(a_0)$.

2. Moderate $|a_0|$ ($1 \times 10^{-12} < |a_0| \leq 0.75$)

This is the most complex case, where the Taylor series expansion is computed for moderate values of $|a_0|$:

1. Initialize odd_coef_0 for odd derivatives of $\text{sinc}(x)$.
2. Loop through each order o up to to , with step size 2.
 - Compute v_1 (coefficient contribution for odd derivatives) and v_2 (even derivatives).

$$v_1 = \sum_{i=0}^6 \frac{1}{\text{odd_coef}[i]} \cdot a_0^{2i+1}, \quad v_2 = \sum_{i=0}^6 \frac{1}{\text{odd_coef}[i]} \cdot a_0^{2i} \cdot (2i+1).$$

- Update $\text{ord_coef}[o]$ and $\text{ord_coef}[o+1]$:

$$\text{ord_coef}[o] = \frac{(-1)^{o/2} \cdot v_1}{o!} * (o+1), \quad \text{ord_coef}[o+1] = \frac{(-1)^{o/2} \cdot v_2}{o!}.$$

- Increment odd_coef using add_odd .
3. Set the zeroth coefficient to $f_0 = \text{sinc}(a_0)$.

3. Small $|a_0|$ ($|a_0| \leq 1 \times 10^{-12}$)

For very small values of $|a_0|$, $\text{sinc}(x)$ is approximated using the series expansion:

$$\text{sinc}(x) \approx 1 - \frac{x^2}{6} + \frac{x^4}{120} - \dots$$

1. Set $\text{ord_coef}[0] = 1$ and $\text{ord_coef}[1] = 0$.
2. Compute higher-order coefficients recursively:

$$\text{ord_coef}[o] = -\frac{\text{ord_coef}[o-2]}{o \cdot (o+1)}.$$

Final Evaluation

Finally, the Taylor series is evaluated using the function `fun_taylor`, which combines the computed coefficients `ord_coef` to generate the output c .

Comparison at $x_0 = 1e-06$

Point: $x_0 = 1e-06$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$4.0981689453125000 \times 10^2$	1
3	$8.5653250000000000 \times 10^4$	2
4	$2.0388565240985107 \times 10^8$	3
5	$2.1989702951932031 \times 10^{11}$	4
6	$3.7529289286551281 \times 10^{14}$	5
7	$4.5035517681883995 \times 10^{15}$	6
8	$4.5035996273631785 \times 10^{15}$	7
9	$4.5035996273703660 \times 10^{15}$	8
10	$4.5035996273704960 \times 10^{15}$	9
11	$4.5035996273704960 \times 10^{15}$	10
12	$4.5035996273704960 \times 10^{15}$	11
13	$4.5035996273704960 \times 10^{15}$	12
14	$4.5035996273704960 \times 10^{15}$	13
15	$4.5035996273704960 \times 10^{15}$	14
16	$4.5035996273704960 \times 10^{15}$	15

Maximum Error:
 $4.5035996273705 \times 10^{15}$ eps,
observed at order 9.
Missing orders are equal to 0.

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.2500000000000000 \times 10^{-1}$	2
3	$0.0000000000000000 \times 10^0$	3
4	$7.8125000000000000 \times 10^{-3}$	4
5	$9.5367431640625000 \times 10^{-7}$	5
6	$1.2207031250000000 \times 10^{-4}$	6
7	$0.0000000000000000 \times 10^0$	7
8	$1.9073486328125000 \times 10^{-6}$	8
9	$0.0000000000000000 \times 10^0$	9
10	$1.4901161193847656 \times 10^{-8}$	10
11	$1.8189894035458565 \times 10^{-12}$	11
12	$2.3283064365386963 \times 10^{-10}$	12
13	$7.1054273576010019 \times 10^{-15}$	13
14	$0.0000000000000000 \times 10^0$	14
15	$1.2490009027033011 \times 10^{-15}$	15

Maximum Error: 1 eps, observed
at order 2.
Missing orders are equal to 0.

Comparison at $x_0 = 0.15$

Point: $x_0 = 0.15$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$7.3437500000000000 \times 10^0$	1
3	$2.7875000000000000 \times 10^1$	2
4	$2.8660156250000000 \times 10^2$	3
5	$1.0381796875000000 \times 10^4$	4
6	$4.2675408569335938 \times 10^4$	5
7	$4.2215365405273438 \times 10^5$	6
8	$1.0488130001253128 \times 10^7$	7
9	$1.2908323054248810 \times 10^7$	8
10	$5.3196699921499765 \times 10^8$	9
11	$1.7305328364951632 \times 10^9$	10
12	$2.3683546289916177 \times 10^9$	11
13	$6.5055108938332361 \times 10^{11}$	12
14	$4.6041114248175205 \times 10^{12}$	13
15	$1.1059598822756793 \times 10^{13}$	14
16	$5.9615893769418775 \times 10^{14}$	15

Maximum Error:
 $5.9615893769419 \times 10^{14}$ eps,
observed at order 15.
Missing orders are equal to 0.

New Approach		
I	Coefficient Error(eps)	Order
1	$0.0000000000000000 \times 10^0$	0
2	$1.9531250000000000 \times 10^{-3}$	3
3	$7.8125000000000000 \times 10^{-3}$	4
4	$6.1035156250000000 \times 10^{-5}$	5
5	$1.2207031250000000 \times 10^{-4}$	6
6	$0.0000000000000000 \times 10^0$	7
7	$1.9073486328125000 \times 10^{-6}$	8
8	$0.0000000000000000 \times 10^0$	9
9	$1.4901161193847656 \times 10^{-8}$	10
10	$0.0000000000000000 \times 10^0$	11
11	$0.0000000000000000 \times 10^0$	12
12	$9.0949470177292824 \times 10^{-13}$	13
13	$4.5474735088646412 \times 10^{-13}$	14

Maximum Error: 1 eps, observed
at order 4.

Missing orders are equal to 0.

Comparison at $x_0 = 0.75$

Point: $x_0 = 0.75$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$5.0000000000000000 \times 10^{-1}$	0
2	$1.1250000000000000 \times 10^0$	1
3	$6.2500000000000000 \times 10^{-1}$	2
4	$1.4843750000000000 \times 10^0$	3
5	$6.1328125000000000 \times 10^{-1}$	4
6	$3.9375000000000000 \times 10^0$	5
7	$3.6298828125000000 \times 10^0$	6
8	$4.5354614257812500 \times 10^0$	7
9	$8.1327590942382812 \times 10^0$	8
10	$8.6326292753219604 \times 10^0$	9
11	$8.2122319638729095 \times 10^0$	10
12	$8.0807279339060187 \times 10^0$	11
13	$1.5716941402642988 \times 10^1$	12
14	$1.0967749557603383 \times 10^0$	13
15	$2.5454915789278402 \times 10^1$	14
16	$6.5027436236090239 \times 10^1$	15

Maximum Error: 66eps,
observed at order 15.
Missing orders are equal to 0.

New Approach		
I	Coefficient Error(eps)	Order
1	$5.0000000000000000 \times 10^{-1}$	0
2	$2.6250000000000000 \times 10^0$	1
3	$2.6875000000000000 \times 10^1$	2
4	$4.3750000000000000 \times 10^{-1}$	3
5	$2.0195312500000000 \times 10^0$	4
6	$1.8554687500000000 \times 10^{-2}$	5
7	$6.0668945312500000 \times 10^{-2}$	6
8	$3.8146972656250000 \times 10^{-4}$	7
9	$9.8991394042968750 \times 10^{-4}$	8
10	$5.2452087402343750 \times 10^{-6}$	9
11	$1.0117888450622559 \times 10^{-5}$	10
12	$4.2840838432312012 \times 10^{-8}$	11
13	$7.1013346314430237 \times 10^{-8}$	12
14	$2.5465851649641991 \times 10^{-10}$	13
15	$3.6288838600739837 \times 10^{-10}$	14
16	$1.1368683772161603 \times 10^{-12}$	15

Maximum Error: 27eps,
observed at order 2.
Missing orders are equal to 0.

Here we are on the border where we switch branch and in the weakest point of our new approach (the farther we go from 0 the worse the accuracy).

From what we presented, it is immediately clear that the new approach is performing very well, especially if we compare it with the previous one ($\frac{\sin(x)}{(x)}$). Moreover we see that it is accurate also very far from 0 (we remind that this is a perturbative approach in 0). Other than that, the new approach is performing well also in the general tests.

3.2.6 Sinh

For Sinh the code and the analysis are literally the same as the one presented for sinc (in the two formulas there is only a $-$ sign as a difference). For this reason, we omit the discussion on Sinh that will be showed in the appendix.

3.2.7 Asinc

Finally, we analyze the Asinc function. Even in this case the accuracy problems were due to the division by x when we are near to 0. We start by describing the code.

```
1 void
2 FUN(asinc) (const T *a, T *c)
3 {
4     assert(a && c); DBGFUN(->);
5     ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6
7     NUM a0 = a->coef[0];
8     NUM f0 = NUMF(asinc)(a0);
9     ord_t to = c->mo;
10
11     if (!to || FUN(isval)(a)) {
12         FUN(setval)(c,f0); DBGFUN(<-); return;
13     }
14
15     if (fabs(a0) > 0.58) { // asinc(x), |x| > 0.58
16         T *t = GET_TMPX(c);
17         FUN(asin)(a,t);
18         FUN(div)(t,a,c);
19         FUN(seti)(c,0,0,f0); // scalar part not very accurate
20         REL_TMPX(t); DBGFUN(<-); return;
21     }
22
23     NUM ord_coef[to+1];
24
25     if (fabs(a0) > 1e-12) { // asinc(x), 1e-12 < |x| <= 0.58
26         FOR(i,to+1) ord_coef[i] = 0;
27
28         enum { ord = DESC_MAX_ORD+1 };
29         static num_t scl_coef[ord+1] = {0};
30         if (!scl_coef[0]) {
31             scl_coef[0] = 1./3;
32             FOR(i,1,ord+1) scl_coef[i] = scl_coef[i-1] * SQR(2*i+1.)/(i*(4*i+6.));
33         }
34
35         num_t fact = 1;
36         num_t tmp_coef[ord+1]; FOR(i,ord+1) tmp_coef[i] = scl_coef[i];
37         for (ord_t o = 1; o <= to; o += 2) {
38             NUM a0pi = 1, dc1, dc2;
39             fact *= o*(o+1.);
40             FOR(i,ord+1) {
41                 tmp_coef[i] *= o > 1 ? CUB(2.*i+o) / (2*i+o+2) : 1;
42                 ord_coef [ o ] += (dc1=a0*SQR(a0pi)*tmp_coef[i]*( o+1 ) / fact);
43                 ord_coef [(o+1)%(to+1)] += (dc2= SQR(a0pi)*tmp_coef[i]*(2*i+1) / fact);
44                 if (fabs(dc1)/fabs(ord_coef[o]) < SQR(DBL_EPSILON) &&
45                     fabs(dc2)/fabs(ord_coef[(o+1)%(to+1)]) < SQR(DBL_EPSILON)) {
46                     if (DBG_SERIES)
47                         printf("asinc: i=%d, o=%d, to=%d, |a0|=%.16e, |dc|=%.16e\n", i, o, to, fabs(a0), MAX(fabs(dc1),fabs(dc2)));
48                     break;
49                 } else if (i == ord) {
50                     if (DBG_SERIES)
51                         printf("asinc: i=%d, o=%d, to=%d, |a0|=%.16e, |dc|=%.16e\n", i, o, to, fabs(a0), MAX(fabs(dc1),fabs(dc2)));
52                     trace(1, "asinc did not converge after %d iterations", i);
53                 }
54                 a0pi *= a0;
55             }
56         }
57         ord_coef[0] = f0;
58     } else { // asinc(x), |x| <= 1e-12
59         ord_coef[0] = 1;
60         ord_coef[1] = 0;
61         for (ord_t o = 2; o <= to; ++o)
62             ord_coef[o] = (ord_coef[o-2] * SQR(o-1.)) / (o*(o+1.));
63     }
64
65     fun_taylor(a,c,to,ord_coef);
66     DBGFUN(<-);
67 }
```

Key Variables and Arrays

- a_0 : The point where the function is evaluated.
- to : The maximum order of the Taylor series expansion.
- `ord_coef`: Array storing the coefficients of the Taylor series.
- `tmp_coef`: Temporary coefficients used in the series computation.
- `mult`: A multiplicative factor dependent on i and o :

$$\text{mult} = \begin{cases} 1, & \text{if } o = 1 \\ \frac{(2i+o)^3}{(2i+o+2)}, & \text{otherwise} \end{cases}$$

Case Analysis

1. Large $|a_0|$ ($|a_0| > 0.58$)

For large absolute values of a_0 :

- Compute $\text{asin}(a)$ directly in the dual space.
- Calculate $f(a) = \frac{\text{asin}(a)}{a}$.
- Set the scalar part of the result (need to reset because of poor accuracy).

2. Moderate $|a_0|$ ($1 \times 10^{-12} < |a_0| \leq 0.58$)

For moderate values of $|a_0|$, the computation involves:

- Initializing temporary coefficients `tmp_coef`.
- Computing coefficients for odd and even derivatives:

$$\frac{\text{Asinc}^{2o+1}(a_0)}{(2o+1)!} = \text{ord_coef}[2o+1] = \sum_{i=0}^{\text{ord}} \frac{a_0^{2i+1} \cdot \text{tmp_coef}[i] \cdot \text{mult}}{(2o+1)!}$$

$$\frac{\text{Asinc}^{2o+2}(a_0)}{(2o+1)!} = \text{ord_coef}[2o] = \sum_{i=0}^{\text{ord}} \frac{a_0^{2i} \cdot \text{tmp_coef}[i] \cdot (2i+1) \cdot \text{mult}}{(2o)!}$$

- Iterative update of tmp_coef using the recurrence relation:

$$\text{tmp_coef}[i] = \text{tmp_coef}[i-1] \cdot \frac{(2i+1)^2}{i(4i+6)} \quad \text{for } i \geq 1$$

- Stopping computation when contributions become negligibly small.

3. Small $|a_0|$ ($|a_0| \leq 1 \times 10^{-12}$)

For very small values of $|a_0|$ we just use the McLaurin expansion of asinc:

- Initialize ord_coef[0] = 1 and ord_coef[1] = 0.
- Compute higher-order coefficients using the recurrence relation:

$$\text{ord_coef}[o] = -\frac{\text{ord_coef}[o-2]}{o \cdot (o+1)}$$

Final Evaluation

The Taylor series is evaluated and given as output.

In the old implementation everything was computed using directly $\text{asinc}(x) = \frac{\text{asin}(x)}{x}$

Comparison at $x_0 = 1e-06$

Point: $x_0 = 1e-06$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$7.068300000000000 \times 10^4$	0
2	$4.4562769862916580 \times 10^{15}$	2
3	$0.000000000000000 \times 10^0$	3
4	$4.5035996273705170 \times 10^{15}$	4
5	$0.000000000000000 \times 10^0$	5
6	$4.5035996273704960 \times 10^{15}$	6
7	$0.000000000000000 \times 10^0$	7
8	$4.5035996273704960 \times 10^{15}$	8
9	$0.000000000000000 \times 10^0$	9
10	$4.5035996273704960 \times 10^{15}$	10
11	$0.000000000000000 \times 10^0$	11
12	$4.5035996273704960 \times 10^{15}$	12
13	$0.000000000000000 \times 10^0$	13
14	$4.5035996273704960 \times 10^{15}$	14

Maximum Error:
 $4.5035996273705 \times 10^{15}$ eps,
observed at order 4.
Missing orders are equal to 0.

New Approach		
I	Coefficient Error(eps)	Order
1	$0.000000000000000 \times 10^0$	0
2	$1.9073486328125000 \times 10^{-6}$	8
3	$0.000000000000000 \times 10^0$	9
4	$1.4901161193847656 \times 10^{-8}$	10
5	$0.000000000000000 \times 10^0$	11
6	$1.1641532182693481 \times 10^{-10}$	12
7	$0.000000000000000 \times 10^0$	13
8	$4.5474735088646412 \times 10^{-13}$	14

Maximum Error: 0eps, observed
at order 3.

Missing orders are equal to 0.

Comparison at $x_0 = 0.15$

Point: $x_0 = 0.15$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$7.068300000000000 \times 10^4$	0
2	$4.4562769862916580 \times 10^{15}$	2
3	$0.000000000000000 \times 10^0$	3
4	$4.5035996273705170 \times 10^{15}$	4
5	$0.000000000000000 \times 10^0$	5
6	$4.5035996273704960 \times 10^{15}$	6
7	$0.000000000000000 \times 10^0$	7
8	$4.5035996273704960 \times 10^{15}$	8
9	$0.000000000000000 \times 10^0$	9
10	$4.5035996273704960 \times 10^{15}$	10
11	$0.000000000000000 \times 10^0$	11
12	$4.5035996273704960 \times 10^{15}$	12
13	$0.000000000000000 \times 10^0$	13
14	$4.5035996273704960 \times 10^{15}$	14

Maximum Error:
 $4.5035996273705 \times 10^{15}$ eps,
observed at order 4.
Missing orders are equal to 0.

New Approach		
I	Coefficient Error(eps)	Order
1	$0.000000000000000 \times 10^0$	0
2	$1.9073486328125000 \times 10^{-6}$	8
3	$0.000000000000000 \times 10^0$	9
4	$1.4901161193847656 \times 10^{-8}$	10
5	$0.000000000000000 \times 10^0$	11
6	$1.1641532182693481 \times 10^{-10}$	12
7	$0.000000000000000 \times 10^0$	13
8	$4.5474735088646412 \times 10^{-13}$	14

Maximum Error: 0eps, observed
at order 3.

Missing orders are equal to 0.

Comparison at $x_0 = 0.58$

Point: $x_0 = 0.58$

Previous Approach		
I	Coefficient Error(eps)	Order
1	$1.0000000000000000 \times 10^0$	0
2	$3.6095620848369598 \times 10^5$	1
3	$4.1225973592005415 \times 10^9$	2
4	$2.5937321806349274 \times 10^{10}$	3
5	$1.5000000000000000 \times 10^1$	4
6	$2.2366795922458699 \times 10^1$	5
7	$1.8331017230369138 \times 10^1$	6
8	$4.7944930492735409 \times 10^0$	7
9	$1.1136944914150037 \times 10^1$	8
10	$1.5921429400796914 \times 10^1$	9
11	$7.4413972989468613 \times 10^0$	10
12	$5.2141973474426608 \times 10^0$	11
13	$6.2057569066466032 \times 10^0$	12
14	$5.8500066418189800 \times 10^0$	13
15	$3.2806512105039980 \times 10^0$	14
16	$1.0145720575982533 \times 10^0$	15

Maximum Error:
25937321807 eps, observed at
order 3.
Missing orders are equal to 0.

New Approach		
I	Coefficient Error(eps)	Order
1	$5.0000000000000000 \times 10^{-1}$	0
2	$2.6250000000000000 \times 10^0$	1
3	$2.6875000000000000 \times 10^1$	2
4	$4.3750000000000000 \times 10^{-1}$	3
5	$2.0195312500000000 \times 10^0$	4
6	$1.8554687500000000 \times 10^{-2}$	5
7	$6.0668945312500000 \times 10^{-2}$	6
8	$3.8146972656250000 \times 10^{-4}$	7
9	$9.8991394042968750 \times 10^{-4}$	8
10	$5.2452087402343750 \times 10^{-6}$	9
11	$1.0117888450622559 \times 10^{-5}$	10
12	$4.2840838432312012 \times 10^{-8}$	11
13	$7.1013346314430237 \times 10^{-8}$	12
14	$2.5465851649641991 \times 10^{-10}$	13
15	$3.6288838600739837 \times 10^{-10}$	14
16	$1.1368683772161603 \times 10^{-12}$	15

Maximum Error: 27 eps,
observed at order 2.
Missing orders are equal to 0.

Here we are at the border where we are switching from one branch to the other one.

After analyzing the performance in the problematic neighborhood of 0, we notice that the new approach is performing significantly well and better than the previous one ($\frac{\text{asin}(x)}{x}$). We are again getting rid of the instability caused by the division. Other than that, the new approach is performing well also in the general tests.

3.2.8 Asinhc

For Asinhc we can literally follow the same analyses we did for asinc since the formulas implemented are the same. There is only a minor adjustment to be made: the derivatives of asinhc have a singularity in 1 and this is causing the lossof accuracy when we are near to 1. This will be analyzed in the Appendix.

Chapter 4

Case Study

In the previous chapter we showed that the new changes are improving a lot the accuracy and stability of our methods for many functions and it is also important to remind that these functions we are working on are the basis for all the code implementing the physics of MAD-NG. Basically we can run all the code with real and complex numbers or dual numbers with no differences in the script. This is also reminding the TPSA is actually the algebra we are using and thus it must be as precise as possible (imagine you cannot rely on multiplication or addition of real numbers). Now we provide a concrete example of the improvements of the library in the framework of MAD-NG for accelerator physics.

4.1 Beam-Beam effect and Twiss parameters

Here we are going to show the improvements made for the library with the specific example of the beam beam effect. Indeed, using the previous implementation of the Faddeeva function had, as a result, a huge error for the β function. This was caused by the fact that the previous implementation of the Faddeeva function was unstable starting from order 1.

4.2 Beam-Beam Effect

The beam-beam effect is a critical phenomenon in high-energy particle colliders, caused by the electromagnetic interaction between charged particle beams. This effect significantly influences beam dynamics, leading to emittance growth, beam instabilities, and potential luminosity degradation. It is particularly important in colliders such as the Large Hadron Collider (LHC) and the Future Circular Collider (FCC).

When two beams pass through each other at the interaction point (IP), the charged particles in one beam experience the collective electromagnetic fields of the opposing beam. This interaction introduces non-linear forces that alter particle trajectories and affect the overall beam dynamics.

4.2.1 Electromagnetic Interaction

Assuming a Gaussian transverse beam distribution, the electromagnetic potential $\Phi(x, y)$ generated by a single beam is given by:

$$\Phi(x, y) = \frac{Ne}{4\pi\epsilon_0\sigma_x\sigma_y} \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right), \quad (4.1)$$

where:

- N is the number of particles per bunch,
- e is the elementary charge,
- ϵ_0 is the vacuum permittivity,
- σ_x and σ_y are the transverse beam sizes in the x and y directions, respectively.

The electric field components derived from this potential are:

$$E_x(x, y) = \frac{Ne}{4\pi\epsilon_0\sigma_x^3\sigma_y} x \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right), \quad (4.2)$$

$$E_y(x, y) = \frac{Ne}{4\pi\epsilon_0\sigma_x\sigma_y^3} y \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right). \quad (4.3)$$

4.2.2 Beam-Beam Force

A particle in the opposing beam with charge q experiences forces due to these fields, given by:

$$F_x = qE_x, \quad F_y = qE_y. \quad (4.4)$$

Substituting the expressions for E_x and E_y , the forces become:

$$F_x(x, y) = \frac{Ne^2}{4\pi\epsilon_0\sigma_x^3\sigma_y} x \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right), \quad (4.5)$$

$$F_y(x, y) = \frac{Ne^2}{4\pi\epsilon_0\sigma_x\sigma_y^3} y \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right). \quad (4.6)$$

These forces act as non-linear focusing or defocusing elements, depending on the relative charges of the beams. For opposite charges, such as in electron-positron colliders, the forces are attractive. In like-charge colliders, such as proton-proton machines, the forces are repulsive.

4.2.3 Beam-Beam Parameter

The strength of the beam-beam interaction is characterized by the beam-beam parameter ξ , defined as:

$$\xi_{x,y} = \frac{r_e N}{4\pi\gamma\sigma_{x,y}(\sigma_x + \sigma_y)}, \quad (4.7)$$

where:

- $r_e = \frac{e^2}{4\pi\epsilon_0 m_e c^2}$ is the classical electron radius,
- γ is the Lorentz factor,
- m_e is the particle mass,
- c is the speed of light.

The beam-beam parameter quantifies the non-linear focusing effect and sets practical limits on beam intensity. If ξ is too high, beam instabilities and significant emittance growth can occur, degrading luminosity.

4.2.4 Impact on Beam Dynamics

The beam-beam effect introduces non-linearities into the Hamiltonian describing the system. The total Hamiltonian, including the beam-beam interaction, is:

$$H = H_0 + \Delta H, \quad \Delta H = -q\Phi(x, y), \quad (4.8)$$

where H_0 represents the unperturbed Hamiltonian. The perturbation ΔH leads to complex dynamics, such as:

- Resonances in the transverse phase space,
- Emittance growth due to non-linear coupling,
- Beam halo formation, increasing particle losses.

4.2.5 Faddeeva Function in Beam-Beam Interactions

The Faddeeva function $w(z)$ provides a sophisticated mathematical approach to modeling electromagnetic interactions in particle beams with non-circular transverse distributions. For elliptical beam configurations, the Faddeeva function allows precise computation of complex electromagnetic field components. In the context of beam-beam effects, the Faddeeva function transforms the beam interaction problem into a complex coordinate space, enabling more accurate calculations of:

- Electromagnetic field components E_x and E_y
- Non-linear focusing effects
- Beam-beam interaction strengths

The transformation involves scaling complex coordinates ζ_{BE} and η_{BE} derived from beam positions and sizes:

$$\zeta_{BE} = \frac{|x| + i|y|}{S}, \quad \eta_{BE} = \frac{\sigma_y/\sigma_x \cdot |x| + i\sigma_x/\sigma_y \cdot |y|}{S} \quad (4.9)$$

where $S = \sqrt{2(\sigma_x^2 - \sigma_y^2)}$, and σ_x and σ_y represent transverse beam sizes. The electromagnetic field components are then calculated using the Faddeeva function:

$$E_x = \text{Im} \left(\frac{1}{2\epsilon_0\sqrt{\pi}S} \left[w(\zeta_{BE}) - w(\eta_{BE}) \exp \left(-\frac{1}{2} \left(\frac{x}{\sigma_x} \right)^2 - \frac{1}{2} \left(\frac{y}{\sigma_y} \right)^2 \right) \right] \right) \quad (4.10)$$

$$E_y = \text{Re} \left(\frac{1}{2\epsilon_0\sqrt{\pi}S} \left[w(\zeta_{BE}) - w(\eta_{BE}) \exp \left(-\frac{1}{2} \left(\frac{x}{\sigma_x} \right)^2 - \frac{1}{2} \left(\frac{y}{\sigma_y} \right)^2 \right) \right] \right) \quad (4.11)$$

where ϵ_0 is the permittivity of vacuum.

This approach provides a rigorous method for handling beam-beam interactions, particularly in scenarios with significant beam size variations or complex beam geometries.

Understanding and mitigating the beam-beam effect is critical for maintaining collider performance. Techniques such as optimizing beam sizes and crossing angles, using crab cavities to correct crossing geometry, and employing active feedback systems are employed to reduce its adverse effects. We now provide a brief explanation of the Twiss parameter to grasp an idea of what we are dealing with

4.3 Twiss Parameters

The Twiss parameters, introduced by Courant and Snyder, provide a convenient way to describe the phase space behavior of particles in a linear accelerator. These parameters are particularly useful when

analyzing beam dynamics near a reference trajectory. They are defined by three functions of position s : $\alpha(s)$, $\beta(s)$, and $\gamma(s)$, which parameterize the oscillatory motion of particles in the transverse plane.

The phase space coordinates can be expressed in terms of the Twiss parameters as:

$$\begin{pmatrix} x \\ x' \end{pmatrix} = \begin{pmatrix} \sqrt{\beta(s)} & 0 \\ -\frac{\alpha(s)}{\sqrt{\beta(s)}} & \frac{1}{\sqrt{\beta(s)}} \end{pmatrix} \begin{pmatrix} x_n \\ x'_n \end{pmatrix},$$

where:

- x and x' are the transverse position and angle of the particle at position s .
- x_n and x'_n are the normalized coordinates, defined by:

$$x_n = \frac{x}{\sqrt{\beta(s)}} \quad \text{and} \quad x'_n = \sqrt{\beta(s)}x' + \frac{\alpha(s)}{\sqrt{\beta(s)}}x.$$

The Twiss parameters have the following interpretation:

- $\beta(s)$: The betatron function, which describes the beam envelope's size at position s .
- $\alpha(s)$: The phase advance, which is related to the tilt of the phase space ellipse.
- $\gamma(s)$: Defined by the relationship $\gamma(s) = \frac{1+\alpha^2(s)}{\beta(s)}$, representing the beam's curvature in phase space.

In the context of the One-Turn Map (OTM), the Twiss parameters allow for a transformation of the beam's dynamics into a normalized phase space. This simplifies the analysis of the beam's motion, which can be represented as a rotation:

$$M = A \cdot R \cdot A^{-1},$$

where:

- A is the transformation matrix containing the Twiss parameters:

$$A = \begin{pmatrix} \sqrt{\beta(s)} & 0 \\ -\frac{\alpha(s)}{\sqrt{\beta(s)}} & \frac{1}{\sqrt{\beta(s)}} \end{pmatrix}.$$

- R is a pure rotation matrix describing the simplified motion:

$$R = \begin{pmatrix} \cos(\Delta\mu) & \sin(\Delta\mu) \\ -\sin(\Delta\mu) & \cos(\Delta\mu) \end{pmatrix},$$

where $\Delta\mu$ is the phase advance over one turn.

The use of Twiss parameters provides a compact description of the beam's dynamics, reducing the complexity of phase space analysis and enabling the characterization of the beam's oscillatory behavior in accelerators.

4.3.1 Numerical Result

We now provide the results of the effects and improvements in MAD-NG thanks to the introduction of 2.19 and the respective perturbative approach. To check that the results are correct we are going to use the PTC code developed by Etienne Forest. First we explain the code used for the simulation.

```

1  local sequence, beam, twiss, option in MAD
2  local quadrupole, drift, wire, beambeam in MAD.element
3  local clight, qelect in MAD.constant
4
5  local qf = quadrupole 'QF' {l=3, k1= 0.04} ! Focusing quadrupole
6  local qd = quadrupole 'QD' {l=3, k1=-0.04} ! Defocusing quadrupole
7  local my_wire = wire 'MyWire' {xma=0.01, yma=0, l=0, l_int=100000, l_phy=1, current=90}
8  local my_bb   = beambeam 'MyBB' {xma=0.01, yma=0, l=0, sigx=1.1e-10, sigy=1e-10, charge=-1, npart=90/(clight*qelect)}
9
10 option.nocharge = true
11
12 local elm = "bbeam" -- "drift" "bbeam" "wire"
13
14 ! Define the FODO cell
15 local fodo = sequence 'FODO' { l=20, refer="entry", beam = beam {},
16   { at= 0 },
17   elm == "wire" and my_wire { at= 5.5 } or
18   elm == "bbeam" and my_bb   { at= 5.5, enabled=true } or
19   elm == "drift" and drift   { at= 5.5 } or nil,
20   qd      { at=10 },
21 }
22
23 local tw = twiss { sequence=fodo, method=2 } --, info=4, debug=4 }
24 tw:write ("tw_"..elm.."_n.tfs")

```

Explanation of Lua Code in MAD-NG

The following Lua code is written for MAD-NG to define and simulate an accelerator sequence. Here's a detailed breakdown:

```
-- Import necessary modules and constants
local sequence, beam, twiss, option in MAD
local quadrupole, drift, wire, beambeam in MAD.element
local clight, qelect in MAD.constant
```

Module and Constant Imports

- **Core Simulation Modules:**

- sequence: Defines accelerator beam line structure
- beam: Manages beam properties
- twiss: Performs beam optics analysis
- option: Configures simulation parameters

- **Beam Elements:**

- quadrupole: Magnetic focusing/defocusing elements
- drift: Spaces between active elements
- wire: Simulates wire-like interactions
- beambeam: Models particle beam interactions

- **Physical Constants:**

- clight: Speed of light
- qelect: Elementary charge

```
local qf = quadrupole 'QF' {l=3, k1= 0.04} -- Focusing quadrupole
local qd = quadrupole 'QD' {l=3, k1=-0.04} -- Defocusing quadrupole
```

Quadrupole Magnet Configuration

- **Focusing Quadrupole (QF):**

- Length: 3 meters
- Focusing strength (k_1): $+0.04 \text{ m}^{-2}$

- **Defocusing Quadrupole (QD):**

- Length: 3 meters

- Defocusing strength (k_1): -0.04

```
local my_wire = wire 'MyWire' {xma=0.01, yma=0, l=0,
                             l_int=100000, l_phy=1, current=90}
local my_bb   = beambeam 'MyBB' {xma=0.01, yma=0, l=0, sigx=1.1e-10,
                                sigy=1e-10, charge=-1, npart=90/(clight*qelect)}
```

Interaction Elements

- **Wire Element (MyWire):**

- Transverse misalignment: (0.01, 0)
- Physical length: 1 meter
- Integration length: 100,000 meters
- Current: 90 amperes

- **Beam-Beam Element (MyBB):**

- Transverse misalignment: (0.01, 0)
- Beam sizes: $\sigma_x = 1.1 \times 10^{-10}$, $\sigma_y = 1.0 \times 10^{-10}$ meters
- Particle charge: -1
- Particle density: $\frac{90}{c \cdot q_e}$

```
option.nocharge = true
local elm = "bbeam" -- Options: "drift", "bbeam", "wire"
```

Simulation Configuration

- Space charge effects: Disabled
- Interaction element type: Beam-beam

```
local fodo = sequence 'FODO' { l=20, refer="entry", beam = beam {},
qf      { at= 0 },
elm == "wire" and my_wire { at= 5.5 } or
elm == "bbeam" and my_bb   { at= 5.5, enabled=true } or
elm == "drift" and drift   { at= 5.5 } or nil,
qd      { at=10 },
}
```

FODO Cell Sequence

- Total length: 20 meters
- Reference point: Entry
- Sequence components:
 - Focusing quadrupole at 0 meters
 - Interaction element at 5.5 meters
 - Defocusing quadrupole at 10 meters
- Flexible element insertion based on `elm` variable

```
local tw = twiss { sequence=fodo, method=2 }  
tw:write ("tw_"..elm.."_n.tfs")
```

Twiss Analysis

- Performs Twiss parameter calculation
- Numerical method: Order 2
- Writes results to file: `tw_<elm>_n.tfs`

The results of the following code for the beta function were totally different with respects to the ones presented by PTC. Indeed for PTC we have

Pos	α_x	β_x	γ_x	μ_x	α_y	β_y	γ_y	μ_y
Start	-5.742653497	119.6560867	0.2839644027	0	0.4401641404	7.691868025	0.1551956516	0
Focusing	8.409855138	110.5324326	0.6489105662	0.003907464564	-0.9497266988	9.060169782	0.2099277219	0.06226512427
Drift	6.787578723	72.53884798	0.6489105662	0.008351590813	-1.474546064	15.12085154	0.2099277219	0.09652317876
BeamBeam	2.873190295	72.53884798	0.127589874	0.008351590813	-0.6585847921	15.12085154	0.09481833247	0.09652317876
Drift	2.299035862	49.26383028	0.127589874	0.02034366347	-1.085267288	22.9681859	0.09481833247	0.1353377536
Defocusing	-3.754902679	53.17319349	0.2839644027	0.03024886376	1.526533702	21.45875292	0.1551956516	0.1556321268
Drift	-5.742653497	119.6560867	0.2839644027	0.04423389456	0.4401641404	7.691868025	0.1551956516	0.2473395301
End	-5.742653497	119.6560867	0.2839644027	0.04423389456	0.4401641404	7.691868025	0.1551956516	0.2473395301

Table 4.1: Twiss parameters obtained through PTC with an equivalent script as the one presented.

Looking at the different values of the Twiss parameters, we immediately notice that the new stable implementation of the Faddeva function within the TPSA world is obtaining, in 4.3.1 the same values obtained through an equivalent simulation using PTC in 4.3.1. The previous implementation instead is not only obtaining wrong values but it is presenting huge and unphysical values.

Pos	α_x	β_x	γ_x	μ_x	α_y	β_y	γ_y	μ_y
Start	0.1339137681	10.22730759	0.09953087736	0	0.7139898941	7.123926248	0.211931106	0
Focusing	0.769833064	7.135875221	0.223188172	0.05360455222	-0.5822938966	6.770289856	0.1977856503	0.07578207114
Drift	0.211862634	4.681635977	0.223188172	0.1247943074	-1.076758022	10.91791965	0.1977856503	0.1227396694
BeamBeam	1.203016577e+16	4.681635977	3.091331518e+31	0.1247943074	-2.865523197e+16	10.91791965	7.209212614e+31	0.1227396694
Drift	-1.391099183e+32	6.259946324e+32	3.091331518e+31	0.6247943074	-3.244145676e+32	1.459865554e+33	7.209212614e+31	0.1227396694
Defocusing	-4.630277553e+32	2.242883517e+33	9.558887056e+31	0.6247943074	-1.494706569e+32	3.08087718e+33	7.251661129e+30	0.1227396694
Drift	-1.132149849e+33	1.340912675e+34	9.558887056e+31	0.6247943074	-2.002322848e+32	5.528797772e+33	7.251661129e+30	0.1227396694
End	-1.132149849e+33	1.340912675e+34	9.558887056e+31	0.6247943074	-2.002322848e+32	5.528797772e+33	7.251661129e+30	0.1227396694

Table 4.2: Twiss parameters obtained by MAD-NG with the previous implementation of the Faddeva function. As one can observe, the values are not only different than the ones obtained through PTC but they are also diverging.

Pos	α_x	β_x	γ_x	μ_x	α_y	β_y	γ_y	μ_y
Start	-5.742653497	119.6560867	0.2839644027	0	0.4401641404	7.691868025	0.1551956516	0
Focusing	8.409855138	110.5324326	0.6489105662	0.003907464564	-0.9497266988	9.060169782	0.2099277219	0.06226512427
Drift	6.787578723	72.53884798	0.6489105662	0.008351590813	-1.474546084	15.12085154	0.2099277219	0.09652317876
BeamBeam	2.873190295	72.53884798	0.127589874	0.008351590813	-0.6585847921	15.12085154	0.09481833247	0.09652317876
Drift	2.299035862	49.26383028	0.127589874	0.02034366347	-1.085267288	22.9681859	0.09481833247	0.1353377536
Defocusing	-3.754902679	53.17319349	0.2839644027	0.03024886376	1.526533702	21.45875292	0.1551956516	0.1556321268
Drift	-5.742653497	119.6560867	0.2839644027	0.04423389456	0.4401641404	7.691868025	0.1551956516	0.2473395301
End	-5.742653497	119.6560867	0.2839644027	0.04423389456	0.4401641404	7.691868025	0.1551956516	0.2473395301

Table 4.3: Twiss parameters obtained by MAD-NG with the new implementation of the Faddeva function. As we can see, we get the same exact values obtained through PTC.

4.4 Summary

We have seen that the new computational approach for the Faddeva function is improving a lot the simulation accuracy in the case of the beam beam effect. Nonetheless, it is important to notice that the benefits of these new functions are not only related to specific cases: when we have a simulation tool, if we know that the mathematics and the numerics tools have been thoroughly tested (so that we know the weak points and the domain of validity of our software), then we can simulate any physics we want without asking ourselves philosophical questions about the validity of our results. On the other hand, using a software whose mathematics has not been tested enough, will generate the need for always having doubts about the validity of our simulation and we will need to double check with other software, experiments etc... We need to remind ourselves that the indeterminacy of a simulation can be originated by the model itself or by the numerics. That is why, if we have a stable numerics we will be able to validate the models we use through the experiments; yet, if the numerics is not stable, then we would not know if the problem we face is within the numerics or the model, and thus, we will need another computational tool independently developed to double check and understand if the problem is related to the model or to the numerics.

Appendix A

Appendix

A.1 Data Generation Code

```
1 (* Define the sinhc function *)
2 sinhc[x_] := Piecewise[{{Sinh[x]/x, x != 0}}, 1]
3
4 (* Define the Eform function *)
5 Eform[x_?NumericQ, ndig_Integer : 20] :=
6 Module[{u, s, p, base, exp, sign, result},
7   u = If[x == 0, 0, x];
8   {s, p} = MantissaExponent[u];
9   If[s != 0, {s = s*10; p = p - 1}];
10  base = ToString[PaddedForm[s, {ndig + 2, ndig}]];
11  exp = If[p >= 0, ToString[p], ToString[-p]];
12  If[StringLength[exp] < 2, exp = StringJoin["0", exp], exp = exp];
13  sign = If[p >= 0, "e+", "e-"];
14  result = StringJoin[base, sign, exp];
15  result]
16
17 (* Compute the series coefficients of sinhc *)
18 data1 = N[
19   Table[SeriesCoefficient[sinhc[x], {x, Pi/3, n}], {n, 0, 15}], 25]
20
21 (* Format the coefficients using Eform *)
22 formattedData = Eform /@ data1
23
24 (* Export the formatted data to a file *)
25 Export["data1.dat", formattedData]
26
27 (* Display the content of the file *)
28 FilePrint["data1.dat"]
```

Listing A.1: Code used to generate data with Mathematica.

This Mathematica code is a general-purpose script designed to compute and format the coefficients of the series expansion of a given function around a specified point. It allows the user to change the function being expanded and the point of evaluation as needed. The steps are as follows:

1. **Define the Function to Expand:** The script defines the function of interest. In this example, the hyperbolic sine cardinal function, $\text{sinhc}(x) = \frac{\sinh(x)}{x}$, is implemented using a piecewise definition to handle the special case where $x = 0$.
2. **Customize Scientific Notation (Eform):** A custom formatting function, `Eform`, is defined to express numeric values in scientific notation with a specified number of significant digits. This ensures the precision and readability of the computed coefficients.
3. **Compute Series Coefficients:** Using the `SeriesCoefficient` function, the code calculates the coefficients of the Taylor series expansion of the defined function around a chosen point (in this example, $x = \frac{\pi}{3}$) up to the 15th term.
4. **Format and Export the Coefficients:** The coefficients are formatted using the `Eform` function and exported to a file named `data1.dat` for storage or further use.
5. **Display Exported Data:** The content of the exported file is printed for verification.

By modifying the function definition and the expansion point in the code, this script can be adapted for a wide range of functions and series expansions, making it a flexible tool for numerical and analytical computations.